

CURSO DE PYTHON

Estruturas de Dados

Apostila passo a passo: da memória RAM ao projeto final

Python 3 · VS Code · estruturas do zero

APOSTILA ELABORADA POR

Professor Celso Brunno Rocha Custódio de Campos

Como esta apostila funciona

Você não pula para o simulador pronto. Em cada aula faz uma estrutura simples, lê o que aquilo significa e só então volta e edita. O projeto final nasce assim: array, lista, pilha, fila, hash, árvore, grafo e heap.

Uso desta apostila

© 2026 Celso Brunno Rocha Custódio de Campos. Material de uso didático.

É vedada a reprodução, distribuição ou comercialização sem autorização do autor.

O aluno pode usar o conteúdo para estudar e montar o projeto do curso.

Sumário

Clique no título para ir à seção.

Antes de começar	5
O que você vai construir	5
Ferramentas	5
Calendário (igual ao site)	5

INSTALAÇÃO

fazer em casa (não entra na aula)	7
Python 3	7
VS Code	7
Conferir se está pronto	7

Se deu erro, faça isto	8
Python não encontrado	8
IndentationError	8
Arquivo na pasta errada	8
NoneType e ponteiro	8

Boas práticas de programação	9
Nomes que explicam	9
Indentação e uma ideia por vez	9
Funções curtas	9
Não se repita	9
Comente o porquê, não o óbvio	9
Não copie sem entender	9
Checklist rápido (todas as aulas)	10

AULA 1

Fundamentos de Memória, Big-O e Arrays	11
Como a RAM guarda um array	11
Big-O em uma frase	12
Live coding: array estático	13

AULA 2

Listas Encadeadas (Linked Lists)	16
Nós e ponteiros	16
Trade-off de Big-O	17
Live coding: lista simples	17

AULA 3

Pilhas (Stacks) e Filas (Queues)	20
---	----

Pilha — LIFO	20
Fila — FIFO	20
Live coding: pilha e fila sobre nós.....	21

AULA 4

Tabelas Hash (Dicionários/Mapas)	24
O problema (antes da técnica).....	24
Resposta em uma frase	24
O dict do Python já é isso.....	24
Peças da tabela hash	25
Inserir e buscar (fluxo mental).....	26
Colisão — o ponto que mais confunde	26
Exercício na mão (antes do código)	27
Live coding: tabela com chaining	27

AULA 5

Árvores Binárias e Árvores de Busca (BST)	29
Vocabulário	29
O nó não é uma lista	29
Live coding: inserção e travessias.....	30
O que acontece se entrar o 35.....	31
Quando a árvore pende.....	32
Fórmulas do balanceamento (AVL)	33
Passo a passo depois de inserir.....	33
Caso 1 — rotação à direita (LL)	34
Caso 2 — rotação à esquerda (RR)	35
Caso 3 — direita no pai, depois esquerda no avô (RL).....	35
Caso 4 — esquerda no pai, depois direita no avô (LR).....	35
Rotação quando o pai já tem filhos	37
Reconstruir pelo meio e inserir o 32.....	37

AULA 6

Grafos e Algoritmos de Travessia	40
O que é (e o que não é)	40
Peças: vértice e aresta.....	41
Não direcionado e direcionado.....	41
Vértices adjacentes e grau	42
Caminhos (e ciclo).....	43
Grafos ponderados	43
Como guardar: lista ou matriz	44
BFS com fila	45

AULA 7

Heaps e preparação para o projeto	48
--	----

Árvore binária completa (anotação)	48
Max-heap e min-heap	49
Isto é heap? (exemplo para anotar).....	49
Onde o heap aparece (só para anotar)	50
Índices no array.....	50
Extrair a raiz, um de cada vez	52
Briefing do Projeto Final	55

AULA 8

Projeto Final (PBL Integrado)	56
O desafio	56
O que a avaliação cobra.....	57
Roteiro da apresentação (5 a 8 minutos)	57
Checklist antes de apresentar.....	58

EXEMPLO PRÁTICO

Huffman junta o curso	59
O que cada aula faz aqui.....	59
O problema (8 bits para tudo)	59
Na mão: contar, juntar, ler	60
Compactar e descompactar	61
Prática.....	62

APÊNDICE

Código de referência	64
Gabarito — Aula 4 (hash).....	64

APÊNDICE

Glossário de funções	66
Índice rápido.....	66

Antes de começar

Esta apostila acompanha o curso Estruturas de Dados. Cada aula tem 2h30. Você lê a explicação, copia o código com o professor e faz a prática no fim.

Usamos o VS Code do primeiro ao último encontro. A instalação é em casa — não consome tempo de aula. O projeto do curso é um simulador de logística: cada estrutura vira uma peça, e a Aula 8 junta o sistema.

O que você vai construir

Não é um script copiado da internet: você constrói em camadas. Uma peça por vez, conferida no checkpoint do fim da aula.

- ▶ Aulas 1 e 2 — memória contígua (array) e memória ligada (lista encadeada).
- ▶ Aulas 3 a 5 — regras de acesso (pilha/fila), busca por chave (hash) e ordem (BST).
- ▶ Aulas 6 e 7 — relações (grafo + BFS) e prioridade (heap).
- ▶ Aula 8 — simulador de logística, README e apresentação.
- ▶ Depois das 8 aulas — Huffman: o mesmo kit (hash + árvore + heap) compactando texto.

Dica

Não tente decorar. Digite o código. Se der erro, leia a mensagem em vermelho — ela quase sempre aponta a linha.

Atenção

Neste curso a `list` e o `dict` nativos do Python existem, mas não são o ponto. Você implementa a estrutura do zero para entender o custo. Usar `lista.append` sem saber o que acontece na memória é nota baixa.

Ferramentas

Python 3 e VS Code. O passo a passo está no capítulo Instalação — fazer em casa. Se a tela falhar, use Se deu erro, faça isto.

Não é um curso de algoritmo do zero. Antes da Aula 1 você precisa conseguir: criar um arquivo `.py`, usar `print`, `if` e `for`, e escrever uma classe simples. Lista e dicionário nativos aparecem só como contraste com a estrutura real. Funções pontuais (`ord`, `deque`, etc.) estão no *Apêndice — Glossário de funções* (nomes em teal sublinhado no texto são links).

Calendário (igual ao site)

- ▶ Aula 1 — Fundamentos de Memória, Big-O e Arrays

- ▶ Aula 2 — Listas Encadeadas (Linked Lists)
- ▶ Aula 3 — Pilhas (Stacks) e Filas (Queues)
- ▶ Aula 4 — Tabelas Hash (Dicionários/Mapas)
- ▶ Aula 5 — Árvores Binárias e Árvores de Busca (BST)
- ▶ Aula 6 — Grafos e Algoritmos de Travessia
- ▶ Aula 7 — Estruturas avançadas (Heaps) e preparação para o projeto
- ▶ Aula 8 — Projeto Final (PBL Integrado)
- ▶ Exemplo prático (no livro) — Huffman junta hash, árvore e heap

INSTALAÇÃO

fazer em casa (não entra na aula)

Atenção

Este capítulo *não* é aula. Nas 8 aulas o professor considera Python e VS Code já instalados. Use estas páginas em casa, antes do curso ou se o computador for novo.

Python 3

Passo 1. Abra python.org e baixe o instalador do Python 3.

Passo 2. No Windows, marque *Add python.exe to PATH* antes de clicar em Install Now.

Passo 3. Termine a instalação. Depois abra o Prompt e digite `python --version`. Deve aparecer 3.x.

VS Code

Passo 1. Baixe em code.visualstudio.com e instale.

Passo 2. Abra o VS Code → ícone de extensões → instale a extensão *Python* (Microsoft).

Passo 3. File → Open Folder na pasta do projeto (a pasta que contém `exemplos/`). Abra um `.py` e rode no terminal: `python exemplos/01_array.py`.

Dica

O terminal precisa estar *na pasta do projeto*. Se aparecer `FileNotFoundError`, você rodou de outra pasta. No VS Code: Terminal → New Terminal (ele abre na pasta que você abriu).

Conferir se está pronto

Passo 1. `python --version` — precisa ser 3.x

Passo 2. `python exemplos/01_array.py` — precisa mostrar o estado da memória e, no fim, um erro de capacidade.

Se deu erro, faça isto

Leia a *última* linha da mensagem em vermelho. Ache o sintoma abaixo e faça o X.

Python não encontrado

Sintoma: `python is not recognized`, comando não encontrado ou o VS Code pede para escolher um interpretador e a lista vem vazia.

- ▶ O Python não está instalado, ou foi instalado *sem* marcar Add python.exe to PATH.
- ▶ Feche o VS Code, instale de novo pelo capítulo de Instalação e *reabra* o programa.
- ▶ No VS Code: Ctrl+Shift+P → *Python: Select Interpreter* → escolha o Python 3.
- ▶ Teste fora do editor: abra o Prompt e digite `python --version`.

Passo a passo da instalação: [Instalação](#) (pág. 7)

IndentationError

Sintoma: `IndentationError: expected an indented block`. O Python usa os espaços da esquerda para saber o que está dentro do `if`, `for` ou `def`.

- ▶ Tudo que pertence ao `if/for/def` deve ficar 4 espaços mais à direita.
- ▶ Não misture Tab com espaço. No VS Code use só espaço (já vem assim).
- ▶ Se copiou código do WhatsApp, cole no editor e realinhe as linhas.
- ▶ A linha logo abaixo de `:` sempre precisa estar mais para dentro.

Arquivo na pasta errada

Sintoma: `FileNotFoundError` ao rodar `python exemplos/01_array.py`.

- ▶ O terminal tem que estar na pasta do projeto (aquela que contém `exemplos/`).
- ▶ No VS Code: File → Open Folder nessa pasta, depois Terminal → New Terminal.
- ▶ Não rode o arquivo de dentro da pasta `Downloads` se o projeto está em outro lugar.

NoneType e ponteiro

Sintoma: `'NoneType' object has no attribute 'proximo'` (ou `dado`, `anterior`). Você andou para além do último nó.

- ▶ Antes de usar `atual.proximo`, teste `if atual is not None`.
- ▶ Lista vazia: a cabeça (`cabeça`) é `None`. Não chame método nela.
- ▶ Esse erro é o equivalente ao `NullPointerException` de outras linguagens.

Boas práticas de programação

Código não é só para o computador. Daqui a uma semana você vai reler o que escreveu hoje. O professor também. Por isso esta apostila cobra hábito, não só o programa que “funciona”.

Nomes que explicam

Use snake_case: `inserir_no_inicio`, `faixa_atual`, `tabela_hash`. Função começa com verbo. Evite `x`, `temp`, `coisa`, `l2`.

Nomes

```
# ruim
n = n.p

# bom
atual = atual.proximo
```

Indentação e uma ideia por vez

Python usa 4 espaços. Uma linha, uma ideia. Não esprema inserção, ponteiro e print na mesma linha.

Funções curtas

Cada operação da estrutura faz uma coisa: inserir, remover, buscar, exibir. O `main` só demonstra. Isso vale da Aula 1 até o projeto.

Não se repita

Copiar o mesmo `while atual is not None` em quatro métodos é armadilha. Quando a lista muda, você corrige um e esquece os outros.

Comente o porquê, não o óbvio

Comentários

```
# ruim
self.topo = novo # atribui topo

# bom
# o novo nó passa a ser o topo: LIFO só mexe nesta ponta
self.topo = novo
```

Não copie sem entender

Atenção

Colar uma BST da internet sem saber apontar a regra esquerda/direita é nota baixa. Se não consegue explicar em voz alta por que escolheu hash em vez de árvore, ainda não é seu código.

Dica

No projeto, o README pede exatamente isso: onde está cada estrutura. Quem escreveu com clareza preenche o README em dois minutos.

Checklist rápido (todas as aulas)

- ▶ Nomes em português ou inglês, mas consistentes.
- ▶ Arquivo `.py` salvo; terminal na pasta do projeto.
- ▶ Erro em vermelho: leia a última linha da mensagem primeiro.
- ▶ Antes de apresentar: rode de novo do zero. O que não roda, não entrega.

AULA 1

Fundamentos de Memória, Big-O e Arrays

Objetivo desta aula (2h30)

Entender alocação contígua na RAM, ler Big-O no pior caso e simular um array estático em Python — inclusive o erro de capacidade cheia.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

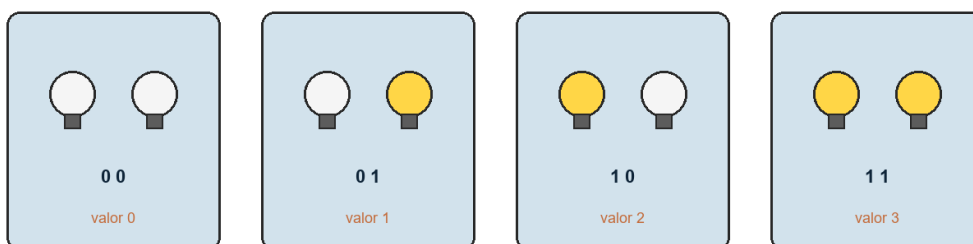
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Como a RAM guarda um array

A memória RAM é um armário com gavetas numeradas (endereços). Um array estático pede um bloco de gavetas vizinhas. Se você declara capacidade 5, o sistema reserva exatamente 5 espaços contíguos.

Antes do array, o aluno precisa ver o tijolo: o bit. 2 bits são 4 combinações (lâmpada ligada/desligada). 32 bits de um `int` clássico têm teto; passar disso é overflow — em linguagens de tamanho fixo. O Python mascara isso; a estrutura de dados, não.

2 bits = 4 combinações = valores 0 a 3



Dois bits: quatro combinações. Cada lâmpada acesa vale um 1.

Quantos valores cabem em N bits?

Bits	Quantidade (2^N)	Sem sinal	Com sinal (aprox.)
2	4	0 a 3	-2 a 1
4	16	0 a 15	-8 a 7
8	256	0 a 255	-128 a 127
16	65 536	0 a 65 535	-32 768 a 32 767
32	4 294 967 296	0 a 4 294 967 295	-2 147 483 648 a 2 147 483 647

Faixas com e sem sinal. Na lousa, complete 8 e 16 bits com a turma antes de mostrar a tabela.

Array precisa de gavetas vizinhas (memoria continua)

lista = [1, 2, 3, 4, 5] vizinho ocupado impede crescer no lugar



Para caber o 6, o computador copia tudo para um bloco novo. Custo $O(n)$.

Cinco slots vizinhos. O bloco rosa já está ocupado: não dá para “empurrar” o 6 ali.

Se precisar do 6º item, a próxima gaveta pode estar ocupada por outro programa. Aí o computador procura um bloco de 6, copia os 5 itens e adiciona o novo. Essa cópia é o custo: tempo linear, $O(n)$.

append quando nao ha vizinho livre: copiar o bloco



O append que parece $O(1)$ às vezes copia a lista inteira. Por isso o “5 segundos / 6 segundos” da lousa.

Atenção

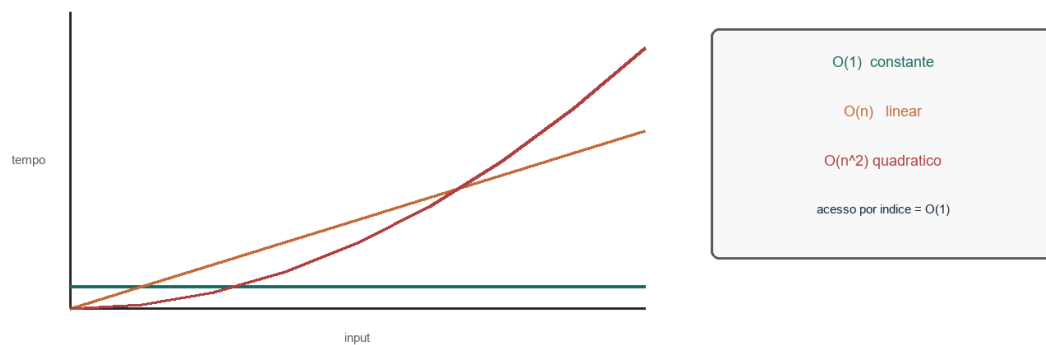
A `list` do Python (`[]`) é um array dinâmico escrito em C. Ela mascara o redimensionamento. Nesta aula você simula o array rígido para ver o limite.

Big-O em uma frase

Big-O descreve o pior caso quando a entrada cresce. Não é “segundos no seu PC”: é como o trabalho cresce.

- ▶ $O(1)$ — constante. Acesso por índice: `endereço_base + (índice * tamanho_do_dado)`.
- ▶ $O(n)$ — linear. Inserir no meio exige empurrar os elementos seguintes.
- ▶ $O(n^2)$ — quadrático. Dois laços aninhados sobre a mesma coleção (aparece como contraste).

Big-O: como o tempo cresce quando o input cresce



O eixo horizontal é o tamanho da entrada; o vertical é o trabalho. $O(1)$ é a reta baixa; $O(n^2)$ dispara.

Live coding: array estático

Crie `exemplos/01_array.py`. Force o `OverflowError`: os alunos precisam ver o erro para entender a rigidez da estrutura.

`exemplos/01_array.py`

```
class ArrayEstatico:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.tamanho_atual = 0
        self.dados = [None] * capacidade

    def acessar(self, indice):
        if 0 <= indice < self.tamanho_atual:
            return self.dados[indice]
        raise IndexError("Índice fora dos limites.")

    def inserir_no_final(self, valor):
        if self.tamanho_atual < self.capacidade:
            self.dados[self.tamanho_atual] = valor
            self.tamanho_atual += 1
        else:
            raise OverflowError("Array cheio.")

    def exibir(self):
        print("Estado da memória:", self.dados)

lista = ArrayEstatico(5)
lista.inserir_no_final(10)
lista.inserir_no_final(20)
lista.inserir_no_final(30)
lista.exibir()
print("posição 1:", lista.acessar(1))
```

Dica

Se `acessar` receber um índice dentro da capacidade mas além de `tamanho_atual`, o slot ainda é `None`. Distinguir “existe na memória” de “já foi preenchido” é o ponto da aula.

Boa prática

Sempre teste o limite: um array que nunca enche não ensina capacidade.

Práticas da aula

Prática A — Sistema de inventário restrito

Vocês foram contratados para o inventário de um jogo clássico. O personagem tem uma mochila com exatamente 4 slots. Dá para adicionar, listar e recusar item extra sem travar o jogo.

exemplos/01_mochila.py

```
class MochilaRPG:
    def __init__(self, slots):
        self.slots = slots
        self.itens_guardados = 0
        self.inventario = [None] * slots

    def coletar_item(self, item):
        print(f"Tentando coletar: {item}...")
        if self.itens_guardados < self.slots:
            self.inventario[self.itens_guardados] = item
            self.itens_guardados += 1
            print(f"[Sucesso] {item} no slot {self.itens_guardados - 1}.")
        else:
            print(f"[Falha] Mochila cheia! Descarte algo para pegar {item}.")

    def abrir_mochila(self):
        print("--- Conteúdo da Mochila ---")
        for i in range(self.slots):
            status = self.inventario[i] or "[Vazio]"
            print(f"Slot {i}: {status}")

mochila = MochilaRPG(4)
for item in ("Poção de Cura", "Espada de Ferro", "Mapa Antigo", "Tocha"):
    mochila.coletar_item(item)
mochila.abrir_mochila()
mochila.coletar_item("Amuleto Mágico")
```

Prática B — Inserir no meio (discussão)

No caderno (não precisa implementar agora): se a mochila tivesse que abrir o slot 0 para um item novo, quantos elementos andam? Isso é $O(n)$. Guarde essa resposta — é o gancho da Aula 2.

Antes da próxima aula, você precisa conseguir

- ▶ Explicar por que o acesso por índice é $O(1)$ e a inserção no meio é $O(n)$.
- ▶ Rodar o array estático e provocar o estouro de capacidade.
- ▶ Dizer o que aconteceria se a mochila ganhasse um “upgrade” de slots usando só array estático.

AULA 2

Listas Encadeadas (Linked Lists)

Objetivo desta aula (2h30)

Quebrar a contiguidade da RAM: montar nós com ponteiros, inserir no início em $O(1)$ e navegar para frente e para trás numa lista dupla.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Retome a pergunta da Aula 1: como aumentar a mochila sem um bloco contínuo novo?

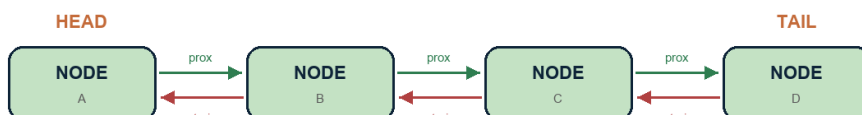
Nós e ponteiros

A lista encadeada espalha os dados em qualquer gaveta livre. Para não se perder, cada nó guarda duas coisas: o dado e o endereço do próximo nó.

- ▶ Simplesmente encadeada — mão única. Cada nó só conhece o próximo.
- ▶ Duplamente encadeada — mão dupla. Conhece o anterior e o próximo (gasta mais memória).

Lista simplesmente encadeada (prox so para a frente)

HEAD aponta o primeiro nó; TAIL o último; o prox do último é None.

Lista dupla: prox (verde) e anterior (vermelho)

Seta verde = próximo. Seta vermelha = anterior. É o custo extra de memória da lista dupla.

Trade-off de Big-O

- ▶ Acesso ao k-ésimo item piora: vira $O(n)$ — é preciso andar nó a nó.
- ▶ Inserção/remoção no início (com o ponteiro na mão) vira $O(1)$: só troca referências.

Live coding: lista simples

exemplos/02_lista.py

```
class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None

class ListaEncadeada:
    def __init__(self):
        self.cabeca = None

    def inserir_no_inicio(self, dado):
        novo = No(dado)
        novo.proximo = self.cabeca
        self.cabeca = novo
        print(f"[{dado}] inserido no início.")

    def exibir(self):
        atual = self.cabeca
        partes = []
        while atual is not None:
            partes.append(str(atual.dado))
            atual = atual.proximo
        print(" -> ".join(partes) + " -> None")

lista = ListaEncadeada()
lista.inserir_no_inicio("Terceiro")
lista.inserir_no_inicio("Segundo")
lista.inserir_no_inicio("Primeiro")
lista.exibir()
```

Dica

Inserir no início inverte a ordem de chegada. “Terceiro, Segundo, Primeiro” no código aparece como Primeiro → Segundo → Terceiro. Mostre isso na lousa.

Práticas da aula

Prática — Reprodutor de mídia

Playlist sem limite prévio de faixas, com botões Próximo e Anterior. Use lista duplamente encadeada.

Cuidado com `None` nas pontas.

ReprodutorStreaming — faixa atual no meio da playlist



A faixa atual (centro) conhece a anterior e a próxima — botões do player.

exemplos/02_reprodutor.py

```

class Faixa:
    def __init__(self, musica):
        self.musica = musica
        self.proximo = None
        self.anterior = None

class ReprodutorStreaming:
    def __init__(self):
        self.faixa_atual = None

    def adicionar_na_fila(self, musica):
        nova = Faixa(musica)
        if self.faixa_atual is None:
            self.faixa_atual = nova
            print(f"Playlist iniciada: {musica}")
            return
        nova.anterior = self.faixa_atual
        nova.proximo = self.faixa_atual.proximo
        if self.faixa_atual.proximo:
            self.faixa_atual.proximo.anterior = nova
        self.faixa_atual.proximo = nova
        print(f"Faixa [{musica}] adicionada como próxima.")

    def proxima_faixa(self):
        if self.faixa_atual and self.faixa_atual.proximo:
            self.faixa_atual = self.faixa_atual.proximo
            print(f">> {self.faixa_atual.musica}")
        else:
            print(">> Fim da playlist.")

    def faixa_anterior(self):
        if self.faixa_atual and self.faixa_atual.anterior:
            self.faixa_atual = self.faixa_atual.anterior
            print(f"<< {self.faixa_atual.musica}")
        else:
            print("<< Início da playlist.")
  
```

```
player = ReprodutorStreaming()
player.adicionar_na_fila("Bohemian Rhapsody")
player.adicionar_na_fila("Stairway to Heaven")
player.adicionar_na_fila("Hotel California")
player.proxima_faixa()
player.proxima_faixa()
player.faixa_anterior()
```

Boa prática

Toda navegação testa `if self.faixa_atual and self.faixa_atual.proximo`. Sem isso, o `NoneType` aparece na hora da demo.

Antes da próxima aula, você precisa conseguir

- ▶ Desenhar três nós no papel e o ponteiro `cabeca`.
- ▶ Inserir no início de uma lista simples e exibir até `None`.
- ▶ Explicar por que o botão Anterior precisa do ponteiro `anterior`.

AULA 3

Pilhas (Stacks) e Filas (Queues)

Objetivo desta aula (2h30)

Restringir o acesso: LIFO na pilha e FIFO na fila. Na live coding as duas nascem sobre nós; na prática (Call Center) usamos `deque` e `list`, ambos em $O(1)$.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em **Instalação** (pág. 7)

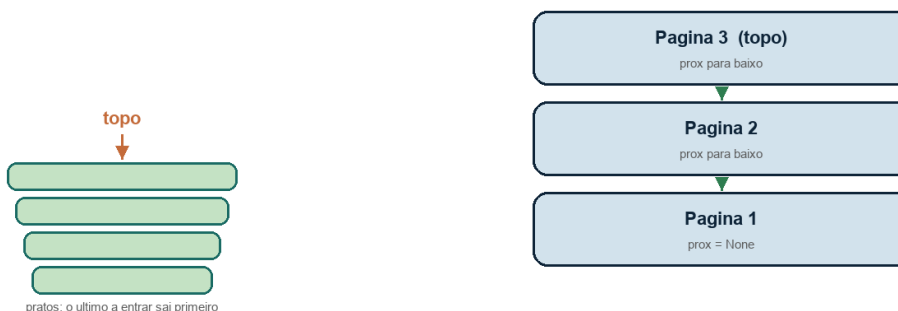
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Nas Aulas 1 e 2 o tema era “onde a memória mora”. Agora o tema é a regra de negócio: de que ponta o dado entra e de que ponta sai.

Pilha — LIFO

Last In, First Out. Analogia: pilha de pratos. Só mexe no topo. Uso real: Ctrl+Z, botão Voltar do navegador, pilha de chamadas do Python.

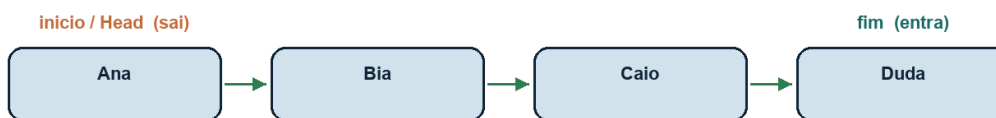
Pilha (LIFO) — so mexe no topo $O(1)$



Pratos e nós: só o topo muda. Empilhar e desempilhar são $O(1)$.

Fila — FIFO

First In, First Out. Analogia: fila de banco. Entra no fim, sai no início. Uso real: impressão, requisições de servidor, matchmaking de jogo.

Fila (FIFO) — entra no fim, sai no início $O(1)$ 

Matchmaking: quem chegou primeiro joga primeiro.

Ana entra primeiro e sai primeiro. No slide da aula isso aparece como Head + deque; aqui a fila nasce nos nós.

Atenção

Usar `lista.pop(0)` numa `list` do Python para simular fila é $O(n)$: todos os itens andam para a esquerda (Aula 1). Por isso a fila da live coding tem ponteiro de início e de fim na lista encadeada. Na prática do Call Center, `deque` resolve as duas pontas em $O(1)$; a `list` do histórico só empilha e desempilha no fim — também $O(1)$.

Live coding: pilha e fila sobre nós

exemplos/03_pilha_fila.py

```

class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None

class Pilha:
    def __init__(self):
        self.topo = None

    def empilhar(self, dado):
        novo = No(dado)
        novo.proximo = self.topo
        self.topo = novo

    def desempilhar(self):
        if self.topo is None:
            raise IndexError("Pilha vazia.")
        dado = self.topo.dado
        self.topo = self.topo.proximo
        return dado

    def vazia(self):
        return self.topo is None

class Fila:
    def __init__(self):
        self.inicio = None
        self.fim = None
  
```

```

def enfileirar(self, dado):
    novo = No(dado)
    if self.fim is None:
        self.inicio = self.fim = novo
    else:
        self.fim.proximo = novo
        self.fim = novo

def desenfileirar(self):
    if self.inicio is None:
        raise IndexError("Fila vazia.")
    dado = self.inicio.dado
    self.inicio = self.inicio.proximo
    if self.inicio is None:
        self.fim = None
    return dado

def vazia(self):
    return self.inicio is None

```

Práticas da aula

Prática – Call Center integrado

Vocês foram contratados para o balcão de um Call Center. Clientes entram numa fila de espera e são atendidos na ordem de chegada. O atendente às vezes encerra o chamado sem querer: precisa de uma pilha de histórico para desfazer o último encerramento e devolver o cliente ao início da fila (não ao fim).



appendleft: Maria volta para o INICIO da fila, não para o fim.

Fila = quem espera (FIFO). Pilha = histórico do atendente (LIFO). Desfazer usa appendleft.

exemplos/03_callcenter.py

```

from collections import deque

class CallCenter:
    def __init__(self):
        self.fila_espera = deque()      # Fila: quem chega
        self.historico_acoes = []       # Pilha: desfazer (Ctrl+Z)

```

```

def novo_cliente(self, nome):
    self.fila_espera.append(nome)
    print(f"[Entrada] Cliente {nome} entrou na fila de espera.")

def atender_proximo(self):
    if len(self.fila_espera) > 0:
        cliente_atendido = self.fila_espera.popleft() # O(1)
        self.historico_acoes.append(cliente_atendido)
        print(f"[Atendimento] Atendendo cliente: {cliente_atendido}")
    else:
        print("[Atendimento] A fila de espera está vazia.")

def desfazer_encerramento(self):
    if len(self.historico_acoes) > 0:
        cliente_recuperado = self.historico_acoes.pop() # O(1)
        self.fila_espera.appendleft(cliente_recuperado)
        print(f"[Desfazer] Erro! {cliente_recuperado} voltou ao início da fila.")
    else:
        print("[Desfazer] Não há ações recentes para desfazer.")

def mostrar_painel(self):
    print(f"\nPainel - Fila atual: {list(self.fila_espera)}")
    print("-" * 30)

sistema = CallCenter()
sistema.novo_cliente("João")
sistema.novo_cliente("Maria")
sistema.novo_cliente("Carlos")
sistema.mostrar_painel()
sistema.atender_proximo()
sistema.atender_proximo()
sistema.mostrar_painel()
sistema.desfazer_encerramento()
sistema.mostrar_painel()
sistema.atender_proximo()

```

Dica

Por que `appendleft()` no desfazer, e não `append`? Se foi erro, a Maria não volta para o fim da fila atrás do Carlos: ela retoma a prioridade de quem já estava sendo atendido.

Antes da próxima aula, você precisa conseguir

- ▶ Dizer em uma frase a diferença LIFO / FIFO apontando a fila e a pilha do Call Center.
- ▶ Explicar por que `popleft()` no `deque` é O(1) e `list.pop(0)` não é.
- ▶ Desfazer o encerramento da Maria e mostrar o painel com ela de volta na frente.

AULA 4

Tabelas Hash (Dicionários/Mapas)

Objetivo desta aula (2h30)

Entender o porquê do hash, mapear chave → índice, tratar colisão por encadeamento e buscar em tempo médio $O(1)$ — sem usar o `dict` nativo no catálogo da aula.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Gancho da Aula 3: o gerente quer o cliente “Carlos” sem percorrer a fila inteira. Array é $O(1)$ por índice, mas o índice não é o CPF. Hash transforma a chave em índice.

O problema (antes da técnica)

Com array, achar `dados[3]` é instantâneo ($O(1)$): você já sabe o endereço. Na vida real a pergunta costuma ser outra: “qual o nome do usuário `u001`?” ou “qual o preço do produto `SKU-9981`?”. Aqui a chave *não* é o índice. Se você só tiver uma lista, precisa percorrer até achar → $O(n)$.

Pergunta da aula: como transformar uma chave qualquer (texto, id, CPF) em um índice rápido?

Resposta em uma frase

Hash = função que transforma a chave em um número; esse número vira o índice de um “balde” na tabela. Depois disso, você só procura *dentro daquele balde*, não na tabela inteira.

Analogia do prédio: imagine 5 andares (baldes 0 a 4). Chega a moradora com a chave “`u001`”. O porteiro (função de hash) calcula: “ela mora no andar 2”. Você sobe *só* o andar 2 e procura o nome na listinha daquele andar. Se duas pessoas caírem no mesmo andar, as duas ficam na listinha — isso é *colisão*. Ainda assim você não vasculha o prédio inteiro.

O dict do Python já é isso

```
peessoa = {"nome": "Bruna", "idade": "20"}  
print(peessoa["nome"]) # rápido — o dict já é hash
```

O `dict` nativo é uma tabela hash (bem otimizada). Nesta aula você *não* usa o `dict` para guardar o catálogo: recria a ideia na mão, para entender o que o Python faz por baixo.

- ▶ `dict` do Python — hash pronto, uso diário.

- ▶ **TabelaHash** da aula — hash didático, para estudar o mecanismo.

Peças da tabela hash

- ▶ **Tabela** — array de tamanho fixo (ex.: 5 ou 8 posições).
- ▶ **Balde** — cada posição é uma listinha (vazia no início).
- ▶ **Função de hash** — transforma a chave em inteiro.
- ▶ **Índice** — $\text{hash}(\text{chave}) \% \text{tamanho}$ (fica entre 0 e $\text{tamanho} - 1$).
- ▶ **Par** — dentro do balde guardamos `[chave, valor]`.

Dica

Balde é o nome padrão (em inglês: *bucket*). Não chamamos só de “posição” porque cada índice é um *recipiente*: pode estar vazio, ter um par ou vários (quando há colisão). Na analogia, “andar” ajuda a imaginar; na estrutura, o nome técnico é balde. Outros textos usam *slot* (endereçamento aberto) ou *bin*.

```
tamanho = 5

baldes:
  [0] []
  [1] []
  [2] [ ["u001", "João"], ["outra", "..."] ] ← colisão
  [3] [ ["u002", "Maria"] ]
  [4] []
```

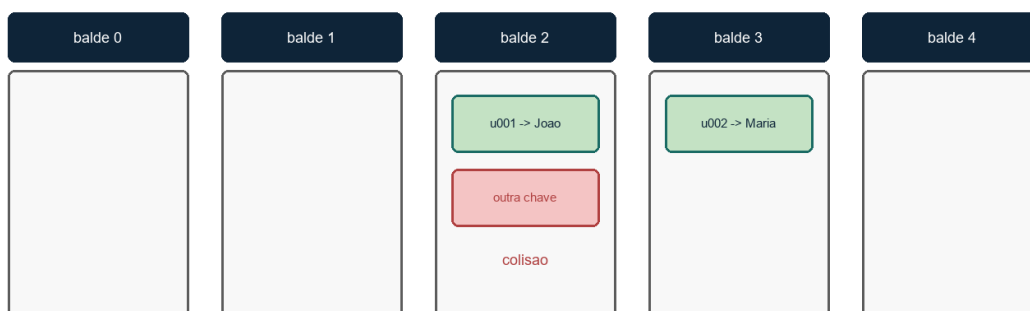
Fórmula didática do curso: $\text{indice} = (\text{soma dos códigos dos caracteres}) \% \text{tamanho}$.

```
def _hash(self, chave):
    # didático: soma dos códigos dos caracteres
    return sum(ord(c) for c in str(chave)) % self.tamanho
```

Dica

[`ord\(\)`](#) devolve o código numérico do caractere; [`sum\(\)`](#) soma esses códigos; [`%`](#) (resto) cabe o índice na tabela. Clique nos nomes para o glossário no final. Essa função de hash é *só para aprender* — a do `dict` real é mais sofisticada. O que importa agora é o fluxo: chave → número → balde → buscar/atualizar.

Tabela hash: $\text{indice} = \text{hash}(\text{chave}) \% \text{tamanho} + \text{colisao por encadeamento}$



Cada balde é uma listinha. Duas chaves no mesmo índice = colisão; as duas ficam no encadeamento.

Inserir e buscar (fluxo mental)

Inserir "u001" → "João":

Passo 1. Calcular `indice = _hash("u001")`.

Passo 2. Ir ao `baldes[indice]`.

Passo 3. Se a chave *já existe* nesse balde → atualiza o valor (não duplica).

Passo 4. Se não existe → `append()` [`chave`, `valor`] na listinha.

Buscar "u001":

Passo 1. Calcular o *mesmo* índice.

Passo 2. Percorrer só aquele balde.

Passo 3. Se achar a chave → devolve o valor; senão → `None`.

Por que é rápido no caso médio? Porque a maioria dos baldes tem poucos itens — você quase não percorre a tabela toda.

Colisão — o ponto que mais confunde

Colisão = duas chaves *diferentes* geram o *mesmo* índice. Isso *não é erro*: é esperado. Tratamento desta aula: *encadeamento (chaining)* — cada balde é uma listinha; as chaves colidentes ficam juntas no mesmo balde (lista da Aula 2 em miniatura).

- ▶ Chaves bem espalhadas, tabela folgada → busca média $\approx O(1)$.
- ▶ Quase tudo no mesmo balde → piora para $O(n)$.

Atenção

Por isso o tamanho da tabela importa. Tabela pequena demais → mais colisões → mais lento. Tempo médio $O(1)$ *assume* tabela folgada e hash espalhada.

Comparação rápida (não misture com as outras aulas)

- ▶ Lista encadeada — inserir no início fácil; busca $O(n)$.
- ▶ Array por índice — $O(1)$ se souber o índice; índice $\neq$ CPF / id.
- ▶ Hash — busca por chave $\approx O(1)$ médio; não mantém ordem; há colisões.
- ▶ BST (Aula 5) — mantém ordem + busca $O(\log n)$; mais complexa.

No projeto final (logística): hash guarda `id_pacote` $\rightarrow$ `dados`; heap decide prioridade; grafo acha a rota. Hash acha a *ficha* do pacote — não decide quem sai primeiro (isso é heap).

Exercício na mão (antes do código)

Use tabela de tamanho 5 e a função do curso: `indice = sum(ord(c) for c in chave) % 5` (veja `ord()`, `sum()` e `%` no glossário). Códigos úteis: `u=117`, `o=48`, `1=49`, `2=50`, `a=97`, `b=98`.

Parte A — Calcular índices

Calcule o índice de `"u001"`, `"u002"`, `"ab"` e `"ba"`. Anote: chave | soma dos ord | soma % 5 | balde.

Parte B — Desenhar a tabela

Insira `"u001"` $\rightarrow$ `"João"` e `"u002"` $\rightarrow$ `"Maria"`. Desenhe os 5 baldes e o conteúdo de cada um.

Parte C — Forçar colisão

Ache duas chaves diferentes no mesmo balde. Sugestão: `"ab"` e `"ba"` têm os mesmos caracteres $\rightarrow$ mesma soma $\rightarrow$ colidem de propósito com esta função didática. Desenhe de novo o balde com dois pares.

Parte D — Busca mental

1) Para buscar `"u001"`, quantos baldes você abre? 2) Se o balde tiver 2 pares, o que você compara? 3) Se o tamanho da tabela for 1, o que acontece com todas as inserções?

Dica

Gabarito no Apêndice — olhe *só depois* de tentar no papel.

Live coding: tabela com chaining

exemplos/04_hash.py

```
class TabelaHash:
    def __init__(self, tamanho=8):
        self.tamanho = tamanho
        self.baldes = [[] for _ in range(tamanho)]

    def _hash(self, chave):
```

```

        return sum(ord(c) for c in str(chave)) % self.tamanho

    def inserir(self, chave, valor):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                par[1] = valor
                return
        self.baldes[indice].append([chave, valor])

    def buscar(self, chave):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                return par[1]
        return None

catalogo = TabelaHash(5)
catalogo.inserir("u001", "João Silva")
catalogo.inserir("u002", "Maria Souza")
print("Busca u001:", catalogo.buscar("u001"))
print("Busca u999:", catalogo.buscar("u999"))

```

Boa prática

Ao inserir, primeiro percorra o balde: se a chave já existe, atualize o valor. Senão a mesma chave aparece duas vezes e a busca pega a antiga.

Práticas no computador

Prática — Catálogo de usuários / cache

Rode `python exemplos/04_hash.py`. Guarde id → nome (ou url → conteúdo). Busque por chave. Mude o tamanho para 5, imprima `catalogo.baldes` após cada inserção, force uma colisão e mostre os dois pares no mesmo balde. Explique em voz alta o que `_hash`, `inserir` e `buscar` fazem — sem olhar o código.

Antes da próxima aula, você precisa conseguir

- ▶ Por que hash existe, se já temos lista?
- ▶ O que significa `hash(chave) % tamanho`?
- ▶ O que é colisão e como o encadeamento resolve?
- ▶ Por que o tempo médio é $O(1)$, e quando isso falha?
- ▶ Em uma frase: diferença entre `dict` nativo e a `TabelaHash` da aula.

AULA 5

Árvores Binárias e Árvores de Busca (BST)

Objetivo desta aula (2h30)

Montar uma BST pela regra esquerda/direita, ver o nó de verdade, inserir o 35, reconhecer quando a árvore pende e equilibrar com as fórmulas AVL e os quatro casos de rotação — sem exigir uma AVL completa no projeto.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

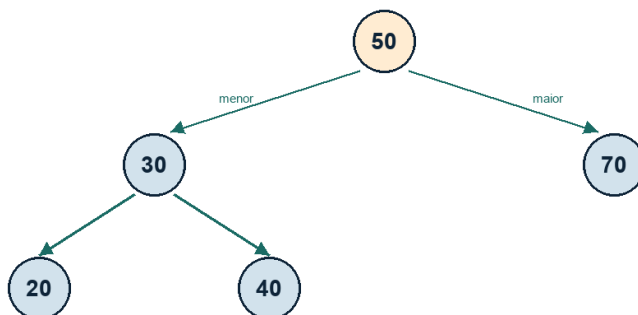
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Hash acha pela chave, mas não entrega os dados ordenados. Árvore organiza hierarquia e, na BST, a busca média cai para $O(\log n)$ se estiver equilibrada.

Vocabulário

- ▶ Raiz — o primeiro nó (entrada da árvore, como a *cabeca* da lista).
- ▶ Nó (*NoArvore*) — objeto com *valor*, *esquerda* e *direita*.
- ▶ Folha — *estado*, não um tipo: nó cujos dois ponteiros são *None*.
- ▶ Árvore binária — cada nó tem no máximo dois filhos. Ainda *não* há regra de ordem.
- ▶ BST — árvore binária com regra: menores à esquerda, maiores à direita.
- ▶ Altura — níveis até a folha mais fundo. Equilibrada $\approx O(\log n)$; vareta = $O(n)$.

BST: esquerda < no < direita insercao 50, 30, 70, 20, 40



em-ordem visita esquerda, raiz, direita -> 20 30 40 50 70

Inserção 50, 30, 70, 20, 40. Em-ordem lê esquerda, o nó, direita: 20 30 40 50 70.

O nó não é uma lista

A BST *não* é uma **list** de folhas. É o mesmo tipo de ligação da Aula 2 — objetos com ponteiros — só que cada nó tem *dois* lados, não um **proximo**.

O nó da BST: valor + dois ponteiros (não é uma lista)



O 50 não aponta para o menor valor da árvore. Aponta para a raiz da subárvore esquerda (30).

- ▶ **esquerda** e **direita** apontam para *filhos*, não para “o menor” e “o maior” da árvore toda.
- ▶ O 20 é menor que o 50, mas o 50 *não* aponta para o 20: aponta para o 30; o 30 aponta para o 20.
- ▶ Por isso a inserção anda nó a nó até achar um **None** e pendurar o novo ali.

Dica

Frase para gravar: BST = nós ligados por dois ponteiros, com a regra menor à esquerda e maior à direita. Folha é o nó que não tem filho.

Live coding: inserção e travessias

exemplos/05_bst.py

```
class NoArvore:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None

class ArvoreBusca:
    def __init__(self):
        self.raiz = None

    def inserir(self, valor):
        if self.raiz is None:
            self.raiz = NoArvore(valor)
        else:
            self._inserir(self.raiz, valor)

    def _inserir(self, no, valor):
        if valor < no.valor:
            if no.esquerda is None:
                no.esquerda = NoArvore(valor)
```

```

        else:
            self._inserir(no.esquerda, valor)
    elif valor > no.valor:
        if no.direita is None:
            no.direita = NoArvore(valor)
        else:
            self._inserir(no.direita, valor)

    def em_ordem(self, no):
        if no:
            self.em_ordem(no.esquerda)
            print(no.valor, end=" ")
            self.em_ordem(no.direita)

    def pre_ordem(self, no):
        if no:
            print(no.valor, end=" ")
            self.pre_ordem(no.esquerda)
            self.pre_ordem(no.direita)

    def pos_ordem(self, no):
        if no:
            self.pos_ordem(no.esquerda)
            self.pos_ordem(no.direita)
            print(no.valor, end=" ")

arvore = ArvoreBusca()
for codigo in (50, 30, 70, 20, 40):
    arvore.inserir(codigo)
print("Em ordem:")
arvore.em_ordem(arvore.raiz)
print("\nPré-ordem:")
arvore.pre_ordem(arvore.raiz)

```

Dica

Em-ordem numa BST imprime crescente. Se não sair crescente, a inserção está errada.

As três travessias não mudam a árvore. Só mudam *quando* você visita o nó (esquerda / eu / direita):

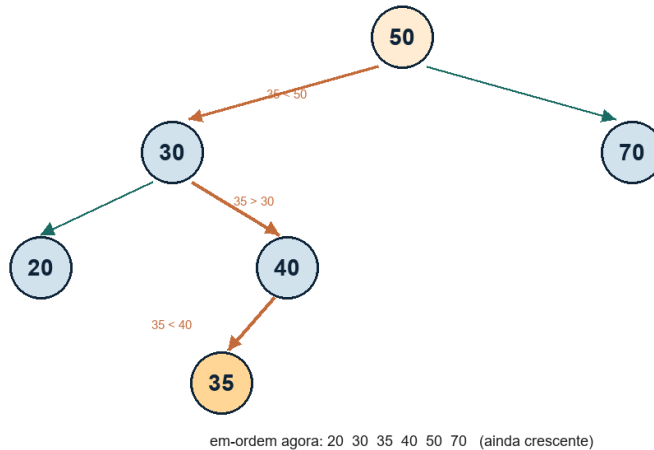
- ▶ Em-ordem — esquerda, eu, direita → 20 30 40 50 70 (prova da BST).
- ▶ Pré-ordem — eu, esquerda, direita → 50 30 20 40 70 (raiz primeiro).
- ▶ Pós-ordem — esquerda, direita, eu → 20 40 30 70 50 (filhos antes do pai).

O que acontece se entrar o 35

O 35 *não* fura no meio da lista e o 50 *não* ganha um terceiro filho. A inserção desce um lado por vez até um ponteiro vazio.

- ▶ $35 < 50 \rightarrow$ vai para a esquerda (nó 30).
- ▶ $35 > 30 \rightarrow$ vai para a direita (nó 40).
- ▶ $35 < 40$ e a esquerda do 40 é `None` $\rightarrow$ nasce o nó 35 ali.

Entra o 35: caminha $50 \rightarrow 30 \rightarrow 40$ e pendura a esquerda do 40



Caminho em laranja: $50 \rightarrow 30 \rightarrow 40$. O 35 vira folha; o 40 deixa de ser folha.

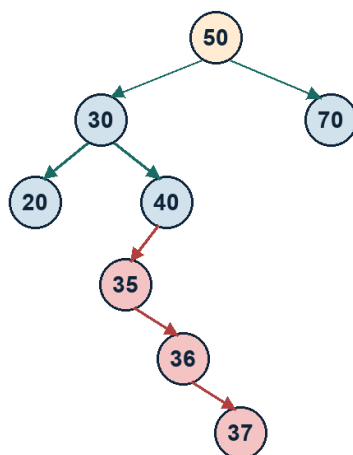
Em-ordem passa a ser **20 30 35 40 50 70**. Sai crescente: a regra continuou valendo.

Quando a árvore pende

Se depois do 35 entram 36 e 37 (sempre um pouco maiores, sempre o mesmo lado), o ramo $35 \rightarrow 36 \rightarrow 37$ vira uma lista. A BST ainda está *correta*. O formato é que ficou ruim: para achar o 37 você visita 50, 30, 40, 35, 36, 37 — não descartou metade.

A BST continua correta, mas vira lista $\rightarrow$ busca $O(n)$

Depois de 35, 36, 37



Regra esquerda/direita ok. Formato ruim: não descarta metade.

Pior caso: inserir 1, 2, 3, 4, 5



Esquerda: 35, 36, 37 no mesmo ramo. Direita: inserir 1, 2, 3, 4, 5 — o pior caso da apostila.

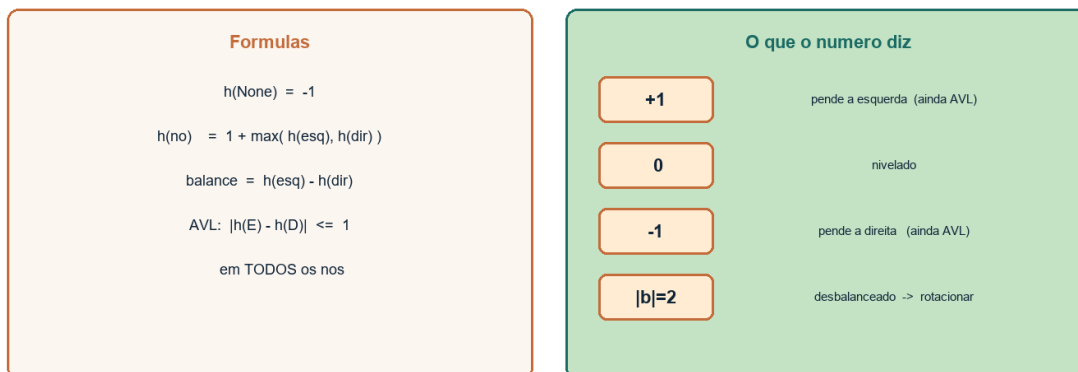
Atenção

BST correta $\neq$ BST rápida. $O(\log n)$ vale se *estiver equilibrada*. Inserir dados já ordenados degenera para $O(n)$ — o custo da lista da Aula 2.

Fórmulas do balanceamento (AVL)

Antes de girar, respire. AVL não é mágica: cada nó guarda (ou calcula) uma *altura*, e o *balance* diz se aquele nó ainda é AVL. Uma árvore $\langle E, \text{raiz}, D \rangle$ é AVL se $|h(E) - h(D)| \leq 1$ e E e D também são AVL — em todos os nós, não só na raiz.

AVL: altura e balance de cada nó



$h(\text{None}) = -1$. Balance positivo pende à esquerda; negativo, à direita. $|b| = 2$ pede rotação.

- ▶ $\text{altura}(\text{None}) = -1$. Folha: $1 + \max(-1, -1) = 0$.
- ▶ $\text{altura}(\text{no}) = 1 + \max(\text{altura}(\text{esquerda}), \text{altura}(\text{direita}))$.
- ▶ $\text{balance}(\text{no}) = \text{altura}(\text{esquerda}) - \text{altura}(\text{direita})$.
- ▶ +1 pende à esquerda, 0 nivelado, -1 pende à direita — *ainda AVL* ($|b| \leq 1$).
- ▶ $|\text{balance}| == 2$ está desbalanceado. Aí sim rotaciona.

altura e balance

```
def altura(no):
    if no is None:
        return -1
    return 1 + max(altura(no.esquerda), altura(no.direita))

def balance(no):
    if no is None:
        return 0
    return altura(no.esquerda) - altura(no.direita)
```

Dica

Pendente (+1 ou -1) não é o mesmo que desbalanceado. Pendente é a iminência. Desbalanceado (+2 ou -2) é o alarme.

Passo a passo depois de inserir

A inserção é a mesma da BST. O extra da AVL começa *depois* do novo nó existir. Nomes fixos (de baixo para cima):

- *z* — o nó que acabou de entrar.
- *y* — o pai de *z*.
- *x* — o avô de *z* (pai de *y*).

Agora, devagar:

Passo 1. Insira como na BST. *z* é o novo nó.

Passo 2. Suba: *y* = *z.pai*, *x* = *y.pai*. (Se não houver avô, não gira ainda.)

Passo 3. Calcule *balance(x)*, *balance(y)*.

Passo 4. Pergunte: *z* nasceu à esquerda ou à direita de *y*? E *y* é filho esquerdo ou direito de *x*?

Passo 5. Se $|\text{balance}(x)| == 2$, escolha o caso 1, 2, 3 ou 4 e gire.

Passo 6. Confira em ordem (continua crescente) e $|\text{balance}| \leq 1$ nos nós que girou.

as duas rotações

```
def rotacionar_direita(x):
    y = x.esquerda
    x.esquerda = y.direita    # x recebe o que estava a direita de y
    y.direita = x
    return y

def rotacionar_esquerda(x):
    y = x.direita
    x.direita = y.esquerda    # x recebe o que estava a esquerda de y
    y.esquerda = x
    return y
```

Na rotação à *direita* de *x*, a esquerda de *x* passa a receber o que estava à *direita* de *y*. Na rotação à *esquerda* de *x*, a direita de *x* passa a receber o que estava à *esquerda* de *y*. É o filho interno — ele não some, muda de pai.

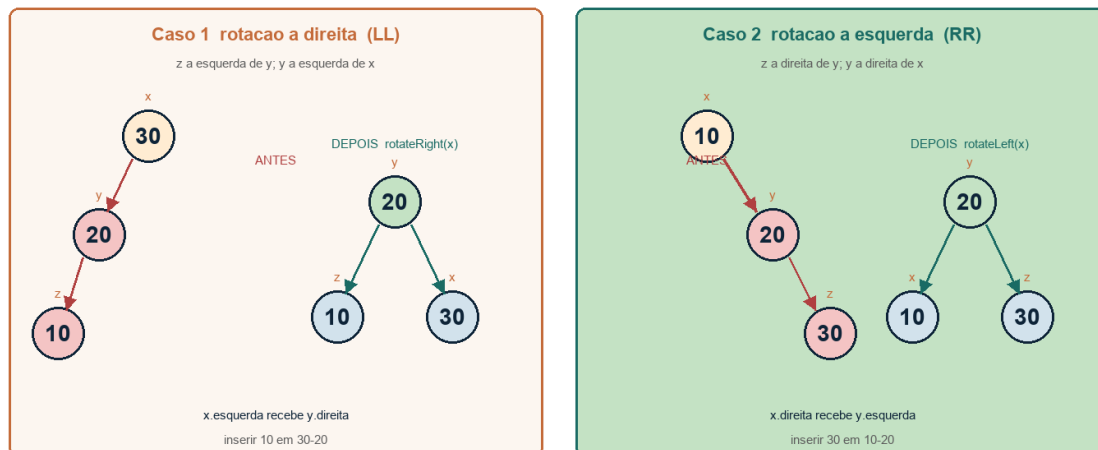
Caso 1 — rotação à direita (LL)

O *z* entrou à *esquerda* do pai. O pai *y* é filho *esquerdo* de um *x* que *já pendia* à esquerda (*balance* == +1). Com o *z*, *balance(x)* vira +2. É uma reta: esquerda-esquerda. Gira só o avô, à direita.

detectar caso 1

```
y = z.pai
x = y.pai
if x.esquerda is y and y.esquerda is z:
    rotacionar_direita(x)
```

Casos 1 e 2: uma rotacao so (reta, nao Z)



Esquerda: caso 1 (10 entra em 30-20). Direita: caso 2 (30 entra em 10-20).

Caso 2 – rotação à esquerda (RR)

Espelho do caso 1. O z entrou à *direita* do pai. O pai y é filho *direito* de um x que *já pendia* à direita ($\text{balance} == -1$). Com o z , $\text{balance}(x)$ vira -2 . Reta direita-direita. Gira só o avô, à esquerda. É a vareta $10 \rightarrow 20 \rightarrow 30$ da aula.

detectar caso 2

```
y = z.pai
x = y.pai
if x.direita is y and y.direita is z:
    rotacionar_esquerda(x)
```

Caso 3 – direita no pai, depois esquerda no avô (RL)

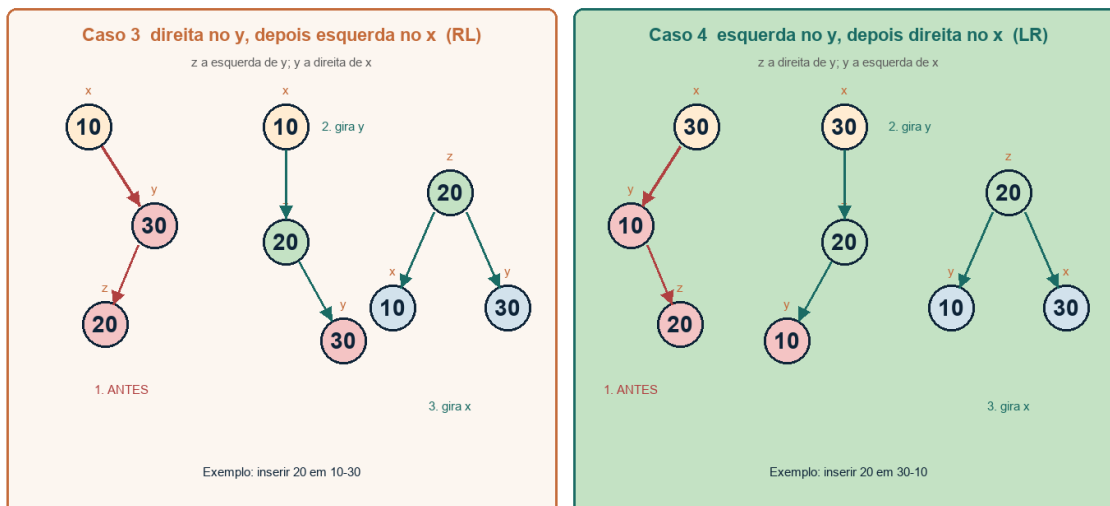
O z entrou à *esquerda* do pai, mas o pai y é filho *direito* de x . O desenho faz um Z (joelho). Uma rotação só no avô *não* basta: primeiro endereita o joelho girando y à direita; depois gira x à esquerda.

- ▶ Exemplo: 10, depois 30, depois 20. $z=20, y=30, x=10$.
- ▶ Passo A — `rotacionar_direita(y)`: o 20 sobe para debaixo do 10.
- ▶ Passo B — `rotacionar_esquerda(x)`: o 20 vira raiz local; 10 e 30 são filhos.

Caso 4 – esquerda no pai, depois direita no avô (LR)

Espelho do caso 3. O z entrou à *direita* do pai, e o pai y é filho *esquerdo* de x . Outro Z. Primeiro gira y à esquerda; depois gira x à direita.

Casos 3 e 4: duas rotações (cenário em Z / joelho)



Caso 3 (inserir 20 em 10-30) e caso 4 (inserir 20 em 30-10). Dois giros cada.

- ▶ Exemplo: 30, depois 10, depois 20. $z=20$, $y=10$, $x=30$.
- ▶ Passo A — `rotacionar_esquerda(y)`.
- ▶ Passo B — `rotacionar_direita(x)`.

exemplos/05_bst_avl.py

```
# z = no recém-inserido, y = pai, x = avô
# ANTES de z, x já pendia (+1 ou -1). DEPOIS, |balance(x)| == 2.
y = z.pai
x = y.pai

# Caso 1 (LL): caminho esquerda-esquerda
if x.esquerda is y and y.esquerda is z:
    rotacionar_direita(x)

# Caso 2 (RR): caminho direita-direita
if x.direita is y and y.direita is z:
    rotacionar_esquerda(x)

# Caso 3 (RL): y a direita de x, z a esquerda de y -> gira y a direita, depois x a esquerda
# Caso 4 (LR): y a esquerda de x, z a direita de y -> gira y a esquerda, depois x a direita
```

Dica

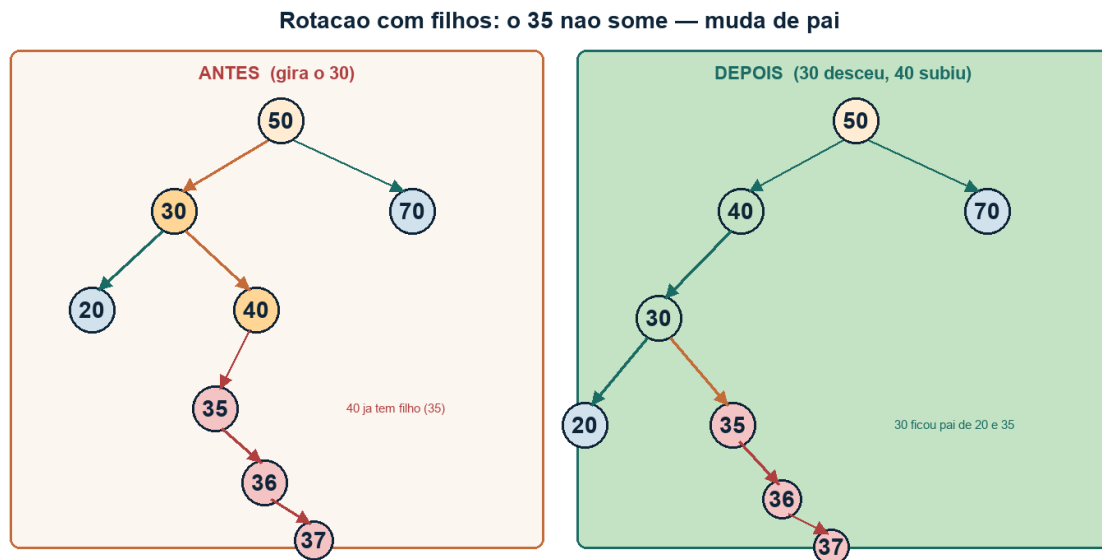
Para lembrar: reta (LL/RR) = um giro no avô. Joelho (LR/RL) = um giro no pai para endereitar, depois o mesmo giro do caso 1 ou 2 no avô.

Leitura extra (mesmos quatro casos e as fórmulas de altura): João Arthur BM, *Árvores Balanceadas (AVL)* — <https://joaoarthurbm.github.io/eda/posts/avl/>.

Rotação quando o pai já tem filhos

A vareta 10-20-30 esconde o filho interno. Na árvore da aula o 30 *já tem* 20 e 40, e o 40 já tem o ramo 35-36-37. Quando você faz o caso 2 no 30, a linha `x.direita = y.esquerda` é o 35 mudando de pai.

Valores 20, 30, 40, 35, 36, 37, 50, 70 (inserção: 50, 30, 70, 20, 40, 35, 36, 37). Gira o 30 à esquerda: o 40 sobe; o 35, que era esquerdo do 40, vira direito do 30.

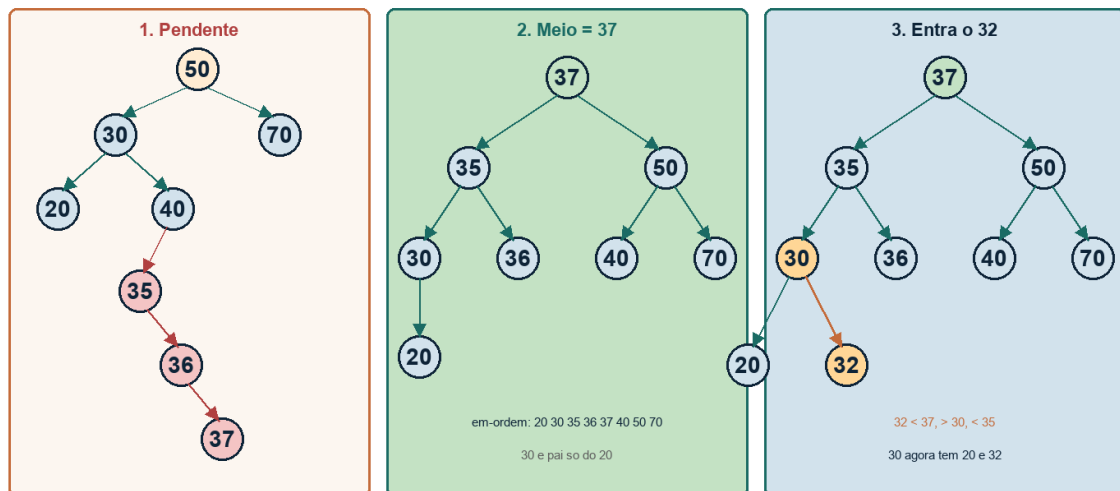


- ▶ Antes: 30 tem 20 e 40; 40 tem 35.
- ▶ Depois: 40 no lugar do 30; 30 tem 20 e 35.
- ▶ Um giro não deixa tudo perfeito — o ramo 35-36-37 ainda pende. AVL olharia o próximo `x`.

Reconstruir pelo meio e inserir o 32

Atalho de sala (não é o algoritmo AVL): pegue a em-ordem e use o do meio como raiz; repita nas metades. Com oito valores o meio é o 37. Depois o 32 entra à direita do 30.

Reset pelo meio, depois entra o 32



Painel 2: 37 no topo. Painel 3: o 32 cai à direita do 30 — o 30 fica com 20 e 32.

`exemplos/05_bst_rebalance.py`

```
arvore.raiz = construir_equilibrada([20, 30, 35, 36, 37, 40, 50, 70])
arvore.inserir(32) # 32 < 37, > 30, < 35 -> direita do 30
```

Atenção

O reset pelo meio equilibra agora. Se depois chegarem 33, 34, 38... em sequência, aquele ramo pende de novo — aí voltam os quatro casos (ou uma AVL de verdade).

Na Aula 8: *usei BST porque preciso de ordem; sei que ids em sequência degeneram; AVL gira com as fórmulas de balance; hash não degenera assim, mas não lista em ordem.*

Práticas da aula

Prática A — Catálogo ordenado / categorias

Insira códigos de produto (ou nomes de categoria comparáveis) e liste em ordem. Opcional: pense no sistema de pastas do computador como analogia de hierarquia — a BST da prática é a versão com regra de busca.

Prática B — Ver a árvore pender (no papel e no código)

No caderno: desenhe 50, 30, 70, 20, 40, depois o 35, depois 36 e 37. Rode `python exemplos/05_bst_pendente.py`. Confira a em-ordem crescente e a rotação 10-20-30 (raiz vira 20).

Prática C — Pai com filhos, reset e o 32

Rode `python exemplos/05_bst_rebalance.py`. Confira: (1) girar o 30 faz o 35 mudar de pai; (2) reconstruir pelo meio deixa o 37 na raiz; (3) o 32 nasce à direita do 30. Desenhe o painel 3 no caderno.

Prática D – Os quatro casos, no papel e no código

Rode `python exemplos/05_bst_avl.py`. No caderno, para cada caso escreva `z`, `y`, `x` e a fórmula `balance = h(esq) - h(dir)`. Marque se foi reta (1 ou 2) ou joelho (3 ou 4).

Antes da próxima aula, você precisa conseguir

- ▶ Desenhar o caminho do 35 até pendurar no 40.
- ▶ Escrever as fórmulas de `altura` e `balance`, e o que `+1`, `0`, `-1` e `|b|=2` significam.
- ▶ Dado `z`, `y` e `x`, dizer se é caso 1, 2, 3 ou 4.
- ▶ Desenhar a rotação do 30: o 35 muda de pai.
- ▶ Depois do reset, dizer onde o 32 entra.
- ▶ Hash vs BST vs AVL: busca, ordem e formato.

AULA 6

Grafos e Algoritmos de Travessia

Objetivo desta aula (2h30)

Entender o grafo como modelo de relações (não como lista nem como fila), ler vocabulário (direção, adjacência, grau, caminho, peso), escolher como guardar e achar caminho com BFS.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

O que é (e o que não é)

Grafo *não* é lista, *não* é fila, *não* é deque e *não* é um jeito de ler uma lista. Grafo é o *modelo*: pontos e ligações. Serve quando o dado não cabe numa fila única — mapa, amizades, ruas, dependências.

```
grafo          = o mapa (pontos + ligações)
lista / matriz = como o mapa é guardado na RAM
fila (deque)   = como a BFS anda no mapa
```

- ▶ *Grafo* — o mapa (quem se liga a quem).
- ▶ *Lista / matriz* — como esse mapa mora na memória.
- ▶ *Fila (deque)* — só a BFS usa, para visitar camada por camada.

Dica

Árvore é um grafo *sem ciclo* e com um começo (raiz). Grafo é o caso geral: pode ter ciclo (Alice–Bruno–Carlos–Alice) e mais de um caminho entre dois pontos.

Grafo da aula: vértices (pessoas) e arestas (amizades)



BFS Gabriel -> Karla (fila): Gabriel-Natan-Erik-Carlos-Brunno-Karla

Vizinhos do Gabriel: Natan e Juan. Juan não entra nesse caminho.

Exemplo da aula: Karla–Brunno–Carlos–Erik–Natan–Gabriel–Juan. Vértices = pessoas, arestas = amizades.

Peças: vértice e aresta

- ▶ **Vértice** (ou nó) — um ponto: pessoa, bairro, CD.
- ▶ **Aresta** — uma ligação entre dois vértices. No não direcionado escrevemos {Alice, Bruno}; no direcionado, CD → Centro.

No projeto final: o grafo é o mapa. Hash é a ficha do pacote. Heap é quem sai primeiro. Sem grafo você tem endereços soltos, sem rua entre eles — pergunta da Aula 8.

Não direcionado e direcionado

Não direcionado (o da aula e da logística simples): a ligação vale nos dois sentidos. Amizade, rua de mão dupla. Se A liga B, B liga A. Por isso o código faz *dois* `append`.

mão dupla

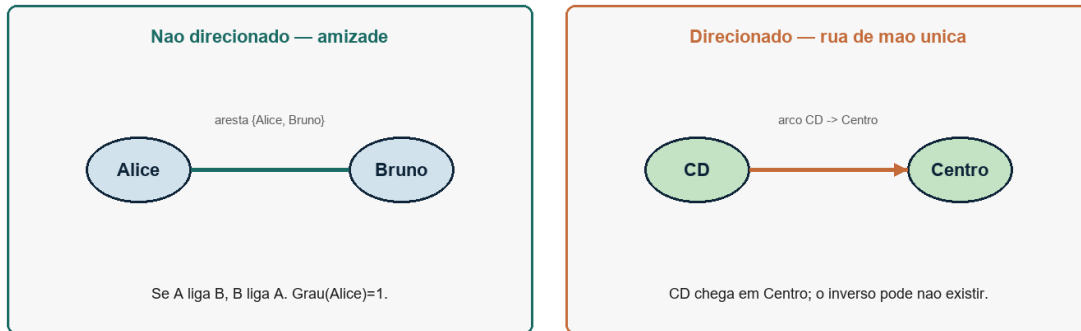
```
def adicionar_aresta(self, a, b):
    self.adicionar_vertice(a)
    self.adicionar_vertice(b)
    self.vizinhos[a].append(b)
    self.vizinhos[b].append(a) # mao dupla
```

Direcionado: a ligação tem seta. Rua de mão única, “seguir” numa rede, voo Congonhas → Galeão. CD chega no Centro; o inverso pode não existir. Só um `append`.

mão única

```
def adicionar_arco(self, origem, destino):
    self.adicionar_vertice(origem)
    self.adicionar_vertice(destino)
    self.vizinhos[origem].append(destino) # so um sentido
    # destino NÃO ganha origem
```

Nao direcionado (mao dupla) vs direcionado (seta)



Esquerda: aresta sem seta. Direita: arco com seta. No direcionado, grau de saída e grau de entrada separam.

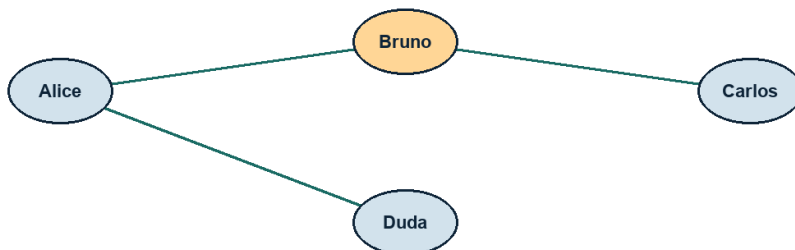
Atenção

O `06_grafo.py` do curso é *não direcionado*. Não copie o segundo `append` se a rua for de mão única — você inventaria o caminho de volta.

Vértices adjacentes e grau

Dois vértices são *adjacentes* (vizinhos) se existe aresta entre eles. *Grau* de um vértice = quantas arestas tocam nele. Na lista de adjacência, `grau(v) = len(self.vizinhos[v])`.

Adjacentes = vizinhos. Grau = quantas arestas tocam o vertice.



Bruno é adjacente a Alice e Carlos. Grau(Bruno)=2. Duda não é adjacente a Bruno.

Bruno é adjacente a Alice e Carlos. Grau(Bruno)=2. Duda não é vizinha de Bruno.

adjacência e grau na lista

```
vizinhos = {
    "Alice": ["Bruno", "Duda"],
    "Bruno": ["Alice", "Carlos"],
    "Carlos": ["Bruno"],
    "Duda": ["Alice"],
}
# Bruno é adjacente a Alice e Carlos
# grau("Bruno") = len(vizinhos["Bruno"]) → 2
```

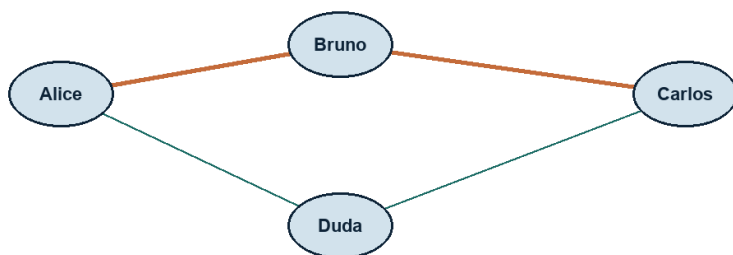
- ▶ Não direcionado: um número só (grau).
- ▶ Direcionado: *grau de saída* (quantas setas saem) e *grau de entrada* (quantas chegam).

- ▶ Folha da árvore da Aula 5 é o vértice de grau 1 (ou 0, a raiz sozinha) — mesma ideia, outro nome.

Caminhos (e ciclo)

Um *caminho* é uma sequência de vértices em que cada um é adjacente ao próximo. Neste curso o caminho *não repete* vértice. O *comprimento* é o número de arestas (Alice–Bruno–Carlos tem comprimento 2).

Caminho = sequência de vértices adjacentes (sem repetir, no curso)



Laranja: Alice-Bruno-Carlos (2 arestas). Teal: Alice-Duda-Carlos (também 2).

Ciclo: Alice-Bruno-Carlos-Duda-Alice. Árvore não tem ciclo; grafo pode ter.

Dois caminhos Alice → Carlos, ambos com 2 arestas. BFS devolve um deles, o de menos arestas.

- ▶ *Caminho* — vai de um ponto ao outro, sem repetir vértice.
- ▶ *Ciclo* — caminho que volta ao começo (Alice–Bruno–Carlos–Duda–Alice). Árvore não tem; grafo pode ter.
- ▶ *Conexo* (não direcionado) — de qualquer um dá para chegar em qualquer outro.

Dica

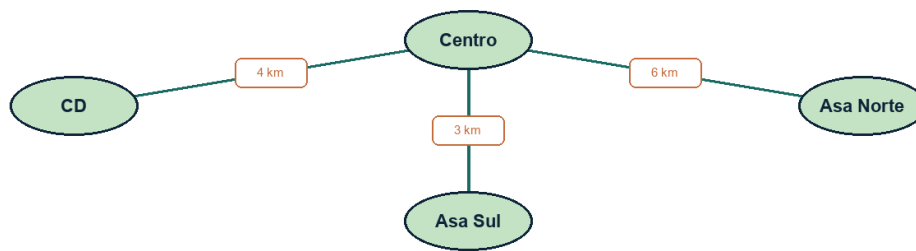
BFS (fila) acha o caminho com *menos arestas*. Se as ruas tiverem km diferentes, menos arestas ≠ menos km — aí o peso entra (mais abaixo).

Grafos ponderados

No grafo *não ponderado* da aula, toda aresta vale 1. No *ponderado*, a aresta carrega um custo: km, minutos, pedágio. Guardamos o par (*vizinho*, *peso*) na listinha.

aresta com km

```
def adicionar_aresta(self, a, b, km):
    self.vizinhos[a].append((b, km))
    self.vizinhos[b].append((a, km))
# CD --4km--> Centro  vira  ("Centro", 4) na listinha do CD
```

Grafo ponderado: a aresta carrega um custo (km, minutos, pedágio)

BFS ignora o km: conta arestas. Caminho mais curto em km pede outro algoritmo (depois do curso).

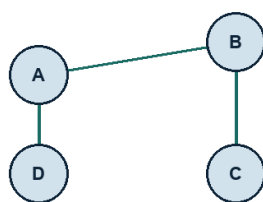
CD—Centro 4 km, Centro—Asa Sul 3 km. BFS ainda conta ruas, não soma km.

Atenção

A BFS desta aula *ignora* o peso. Caminho mais curto em km é outro algoritmo (Dijkstra) — fora do recorte de 2h30. No projeto, BFS no mapa não ponderado já vale.

Como guardar: lista ou matriz

O grafo é a ideia. Na RAM você escolhe uma *representação*. As duas guardam o *mesmo* mapa; mudam custo de memória e de consulta.

Mesmo grafo, duas representacoes**Lista de adjacencia**

A: [B, D]

B: [A, C]

C: [B]

D: [A]

Curso usa esta.

Grafo esparsos: poucas arestas.

Matriz V x V

	A	B	C	D
A	0	1	0	1
B	1	0	1	0
C	0	1	0	0
D	1	0	0	0

Olhar aresta é $O(1)$. Gasta $O(V^2)$.

Lista: para cada vértice, os vizinhos. Matriz: tabela $V \times V$ com 0/1 (ou o peso).

matriz do grafo A-B-C-D

```

#           Alice Bruno Carlos Duda
# Alice     0       1       0       1
# Bruno     1       0       1       0
# Carlos     0       1       0       0
# Duda       1       0       0       0
# 1 = tem aresta. Olhar a célula é  $O(1)$ . A tabela inteira gasta  $O(V^2)$ .
  
```

- *Lista de adjacência* — o curso. Boa quando o grafo é *esparso* (poucas ruas). Olhar os vizinhos de v é percorrer `vizinhos[v]`.

- *Matriz de adjacência* — tabela $V \times V$. “Existe aresta A–C?” é $O(1)$. Gasta $O(V^2)$ mesmo se quase tudo for zero.

Dica

`self.vizinhos` no código é um `dict` de listinhas — hash (Aula 4) apontando para listas (Aula 2). O *grafo* é o que isso representa, não o `dict` em si.

BFS com fila

Busca em largura visita camada por camada. A estrutura certa é a *fila* da Aula 3 (quem entra primeiro na visita sai primeiro). O caminho mais curto em *número de arestas* cai de graça se você guardar `veio_de`.

Passo 1. Enfileira o início. Marca `veio_de[inicio] = None`.

Passo 2. `popleft` no atual. Se for o destino, para.

Passo 3. Para cada *adjacente* ainda não visto: marca `veio_de[vizinho] = atual` e enfileira.

Passo 4. No fim, anda `veio_de` de trás para frente e inverte — esse é o caminho.

exemplos/06_grafo.py

```
from collections import deque

class Grafo:
    def __init__(self):
        self.vizinhos = {}

    def adicionar_vertice(self, nome):
        self.vizinhos.setdefault(nome, [])

    def adicionar_aresta(self, a, b):
        self.adicionar_vertice(a)
        self.adicionar_vertice(b)
        self.vizinhos[a].append(b)
        self.vizinhos[b].append(a)

    def bfs(self, inicio, destino):
        fila = deque([inicio])
        veio_de = {inicio: None}
        while fila:
            atual = fila.popleft()
            if atual == destino:
                break
            for vizinho in self.vizinhos[atual]:
                if vizinho not in veio_de:
                    veio_de[vizinho] = atual
                    fila.append(vizinho)
        if destino not in veio_de:
            return None
        caminho = []
```

```

    atual = destino
    while atual is not None:
        caminho.append(atual)
        atual = veio_de[atual]
    caminho.reverse()
    return caminho

rua = Grafo()
rua.adicionar_aresta("Karla", "Brunno")
rua.adicionar_aresta("Brunno", "Carlos")
rua.adicionar_aresta("Carlos", "Erik")
rua.adicionar_aresta("Erik", "Natan")
rua.adicionar_aresta("Natan", "Gabriel")
rua.adicionar_aresta("Juan", "Gabriel")
print("Vizinhos do Gabriel:", rua.vizinhos["Gabriel"])
print("Caminho Gabriel → Karla:", rua.bfs("Gabriel", "Karla"))

```

Dica

Aqui o exemplo usa [deque](#) + [popleft\(\)](#) para a BFS ficar $O(1)$ na ponta da fila. É a mesma regra FIFO que vocês implementaram com nós. No projeto, pode colar a classe [Fila](#) da Aula 3 no lugar do deque. Vértices novos usam [setdefault\(\)](#). DFS usaria *pilha* — não garante menos arestas.

Práticas da aula

Prática A — No papel, o vocabulário

Desenhe 5 pessoas e algumas amizades. Marque: um par adjacente, o grau de cada um, um caminho de 2 arestas, um ciclo se houver. Diga se o desenho é direcionado ou não.

Prática B — Rede social restrita

Use o grafo da aula ([Karla ... Juan](#)) no [06_grafo.py](#). Mostre os vizinhos de Gabriel, amigos em comum (interseção das listas) e o caminho Gabriel → Karla com BFS. Confira com o desenho.

Prática C — Mão única (no caderno)

Pegue CD → Centro e *não* desenhe a volta. Escreva a lista de adjacência. O que muda se alguém pedir o caminho Centro → CD?

Antes da próxima aula, você precisa conseguir

- ▶ Em uma frase: grafo ≠ lista ≠ fila. O que cada um faz nesta aula?
- ▶ Diferença não direcionado / direcionado no [append](#).
- ▶ O que é vértice adjacente e como lê o grau na lista.

- ▶ O que é caminho e por que ciclo existe no grafo e não na BST.
- ▶ Quando a matriz ganha da lista — e por que o curso usa lista.
- ▶ Por que BFS usa fila (e não pilha) para menos arestas.

AULA 7

Heaps e preparação para o projeto

Objetivo desta aula (2h30)

Manter o menor (ou maior) elemento no topo com um heap em array, usar isso como fila de prioridade e receber o briefing do projeto final.

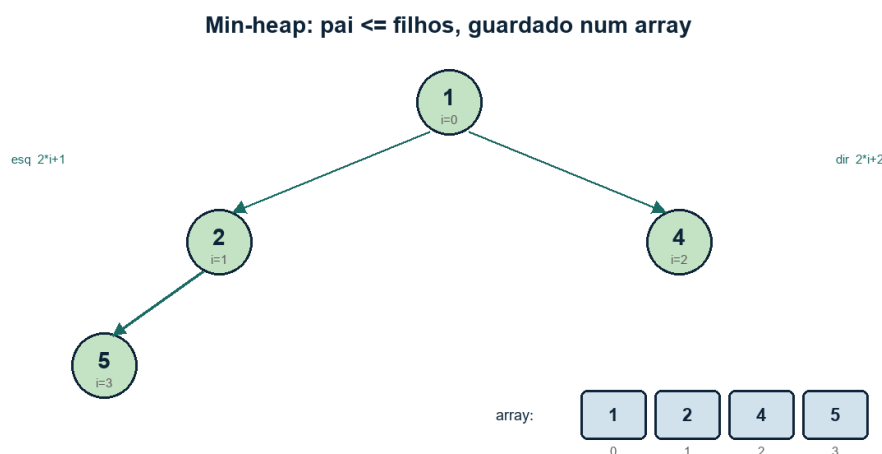
Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Fila comum atende por ordem de chegada. Hospital, suporte e entrega urgente atendem por gravidade. Heap é a árvore (quase) completa guardada num array, em que $\text{pai} \leq \text{filhos}$ (min-heap).

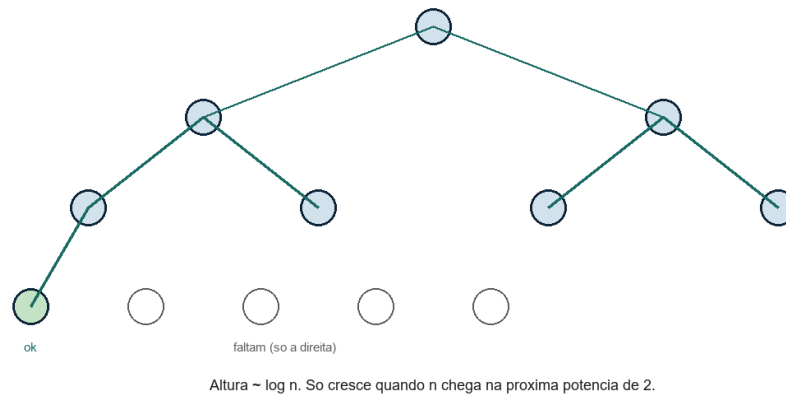


A árvore e o array são a mesma estrutura. Filho esquerdo de i está em $2*i+1$.

Árvore binária completa (anotação)

Heap *não* é a BST da Aula 5. A forma é uma *árvore binária completa*: cada nível enche da *esquerda para a direita*, e um nível só começa quando o de cima está cheio. Podem faltar nós — só no último nível, e só à *direita*.

Árvore binária completa: enche da esquerda. So o ultimo nivel pode falhar, a direita.



Completa = preenchida de cima para baixo, da esquerda para a direita. É isso que autoriza o array.

- ▶ Altura $\approx \log n$. Só aumenta quando n chega na próxima potência de 2.
- ▶ Sem essa regra, as fórmulas $2*i+1$ / $2*i+2$ não batem: o filho esquerdo deixaria de cair no lugar fixo do vetor.
- ▶ BST pendente (Aula 5) *não* é completa — por isso não mora num array com essas contas.

Max-heap e min-heap

Além de ser completa, a heap tem uma regra de *valor*. São duas variantes; muda só a comparação. O código da aula é *min-heap*.

- ▶ *Min-heap* — pai $\leq$ filhos. O *menor* está na raiz (índice 0). É o da triagem e do [exemplos/07_heap.py](#).
- ▶ *Max-heap* — pai $\geq$ filhos. O *maior* está na raiz. É o exemplo `[8, 6, 7, 1, 3, 4, 2]` mais abaixo.

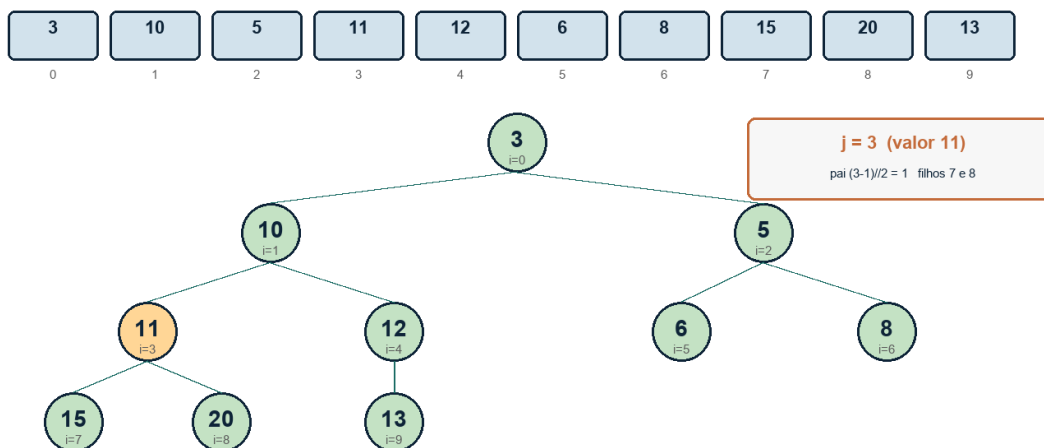
Dica

Os dois são heap. Se o slide disser “pai $\geq$ filhos”, é máximo. Se o Python usar `<` na hora de subir, é mínimo. Não misture a regra na mesma árvore.

Isto é heap? (exemplo para anotar)

Array `[3, 10, 5, 11, 12, 6, 8, 15, 20, 13]`. É árvore *completa*. Pegue o nó $j = 3$ (valor 11): pai $(3-1)//2 = 1$ (o 10); filhos $2*3+1 = 7$ e $2*3+2 = 8$ (15 e 20). Em todo pai, o valor é $\leq$ os filhos — então é min-heap. *Não* é max-heap: a raiz é 3, o menor, não o maior.

Exemplo: completa + min-heap. Array e árvore são a mesma coisa.



Mesmos dez números no vetor e na árvore. Destaque: índice 3 → pai 1, filhos 7 e 8.

Onde o heap aparece (só para anotar)

Heap é a estrutura por trás da *fila de prioridade*: o próximo a sair não é quem chegou primeiro, é o de maior (ou menor) prioridade. Exemplos — o curso só cobra os dois primeiros:

- ▶ *Neste curso* — triagem (menor gravidade sai primeiro) e pacote urgente na logística.
- ▶ *Compactação (Huffman)* — o algoritmo vai pegando os dois símbolos *menos* frequentes; isso é min-heap. A árvore de códigos que nasce no fim *não* é heap. O modelo completo está no capítulo *Exemplo prático — Huffman junta o curso*.
- ▶ *Grafos* — Dijkstra e Prim usam heap para escolher o próximo vértice mais barato (a BFS da Aula 6 usa *fila*, porque toda aresta vale 1).
- ▶ *Busca A** — IA / jogos: o próximo a expandir é o de menor estimativa de custo.
- ▶ *Estatística* — manter os k maiores (ou menores) de um fluxo, sem ordenar tudo.
- ▶ *Sistema operacional* — quem o processador atende agora (prioridade, interrupção).

Dica

Na apresentação da Aula 8: “usei heap porque a urgência não é FIFO”. Não precisa citar Dijkstra. Se citarem Huffman, digam: heap escolhe o próximo; compactar o arquivo é outro passo — o capítulo prático no fim do livro mostra as três peças juntas.

Índices no array

- ▶ Filho esquerdo de i : $2*i + 1$. Filho direito: $2*i + 2$. Pai: $(i - 1) // 2$.
- ▶ Inserir: coloca no fim e “sobe” trocando com o pai enquanto for menor.
- ▶ Extrair o topo: pega o índice 0, põe o último no lugar e “desce”.

exemplos/07_heap.py

```
class MinHeap:
```

```

def __init__(self):
    self.dados = []

def _pai(self, i):
    return (i - 1) // 2

def _esq(self, i):
    return 2 * i + 1

def _dir(self, i):
    return 2 * i + 2

def _sobe(self, i):
    while i > 0:
        p = self._pai(i)
        if self.dados[i] < self.dados[p]:
            self.dados[i], self.dados[p] = self.dados[p], self.dados[i]
            i = p
        else:
            break

def _desce(self, i):
    n = len(self.dados)
    while True:
        menor = i
        e, d = self._esq(i), self._dir(i)
        if e < n and self.dados[e] < self.dados[menor]:
            menor = e
        if d < n and self.dados[d] < self.dados[menor]:
            menor = d
        if menor == i:
            break
        self.dados[i], self.dados[menor] = self.dados[menor], self.dados[i]
        i = menor

def inserir(self, valor):
    self.dados.append(valor)
    self._sobe(len(self.dados) - 1)

def extrair_min(self):
    if not self.dados:
        raise IndexError("Heap vazio.")
    minimo = self.dados[0]
    ultimo = self.dados.pop()
    if self.dados:
        self.dados[0] = ultimo
        self._desce(0)
    return minimo

heap = MinHeap()
for n in (5, 1, 4, 2):
    heap.inserir(n)
print("Extrai em ordem:", heap.extrair_min(), heap.extrair_min(), heap.extrair_min())

```

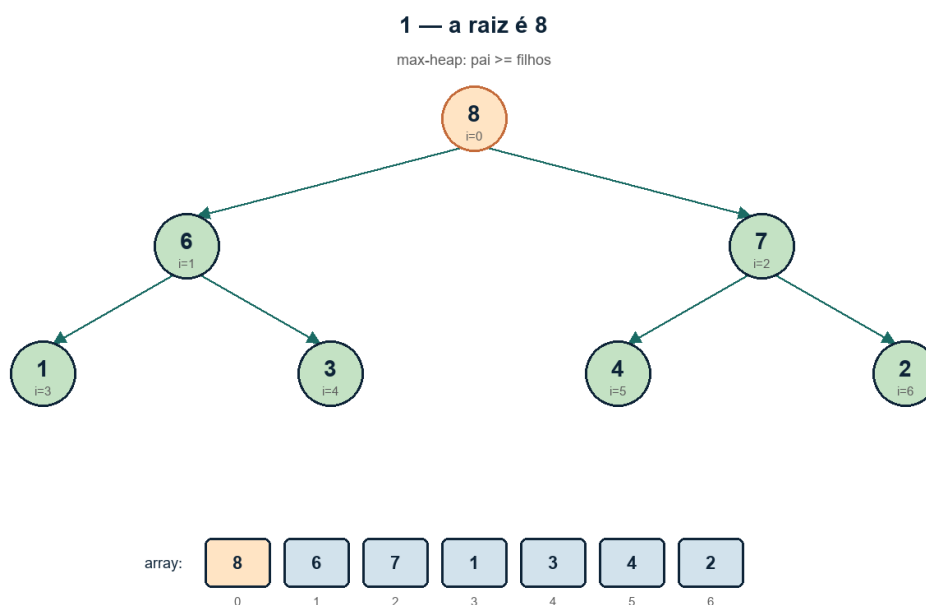
Dica

No Python do dia a dia existe `heapq` (min-heap nativo). Mostre um `heappush` / `heappop` no quadro depois que o heap próprio funcionar. No projeto, vale a estrutura que você souber defender — a do zero prova que entendeu o índice.

Extraír a raiz, um de cada vez

Ler o array da esquerda para a direita não é a ordem de saída. A lista `[8, 6, 7, 1, 3, 4, 2]` já é uma max-heap: pai $\geq$ filhos, e o maior está no índice 0. No código desta aula a comparação é a inversa (min-heap). A casa que muda é a mesma.

Ao tirar a raiz, o último índice ocupa o índice 0 e a lista encolhe no fim. Esse valor troca com o filho maior ($2*i+1$ ou $2*i+2$) enquanto o filho for maior. Os outros índices ficam parados. Não é um escorregão para a casa vizinha.

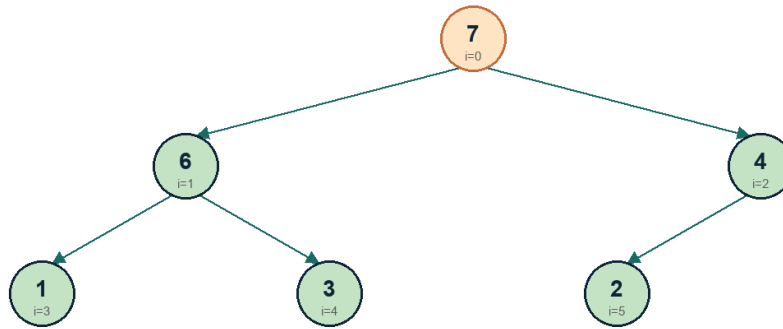


1. A raiz é 8. O 4 está debaixo do 7, no índice 5.

saiu
8

2 — saiu o 8. A nova raiz é 7

max-heap: pai $\geq$ filhos

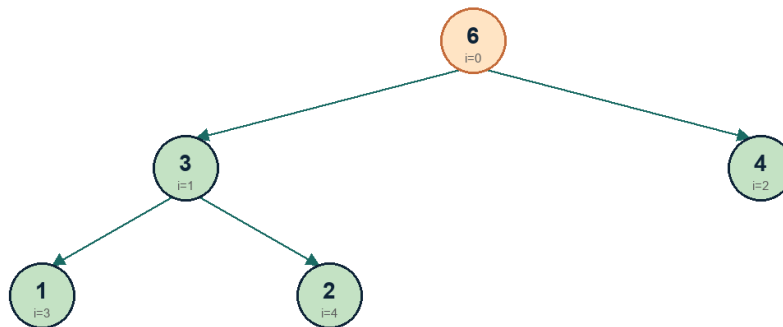


2. Saiu o 8. O 2 foi para o índice 0, trocou com o 7 e depois com o 4. Nova raiz: 7.

saiu
7

3 — saiu o 7. A nova raiz é 6

max-heap: pai $\geq$ filhos

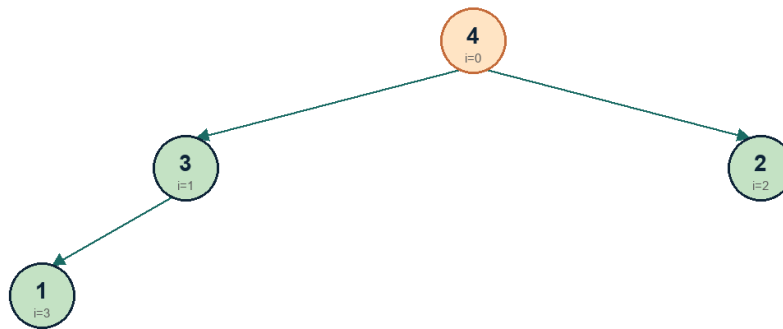


3. Saiu o 7. O 2 trocou com o 6 e depois com o 3. O 4 continua à direita. Nova raiz: 6.

saiu
6

4 — saiu o 6. A nova raiz é 4

max-heap: pai $\geq$ filhos



array:

4	3	2	1
0	1	2	3

4. Saiu o 6. O 4 sobe do índice 2 para o 0 e vira a raiz. A ordem de saída foi 8, 7, 6, 4.

Práticas da aula

Prática — Triagem hospitalar

Pacientes entram com um número de gravidade. Quem tem o menor número (mais crítico) sai primeiro, mesmo tendo chegado depois. Use o heap. O contador `ordem` desempata quem tem a mesma gravidade (FIFO entre iguais).

exemplos/07_triagem.py

```

class Triagem:
    def __init__(self):
        self.heap = MinHeap()
        self.ordem = 0

    def chegar(self, gravidade, nome):
        # tupla: menor gravidade numérica = mais urgente (1 = crítico)
        self.heap.inserir((gravidade, self.ordem, nome))
        self.ordem += 1
        print(f"{nome} entrou com gravidade {gravidade}.")

    def atender(self):
        gravidade, _, nome = self.heap.extrair_min()
        print(f"Atendendo {nome} (gravidade {gravidade})")

hospital = Triagem()
hospital.chegar(5, "Paciente leve")
hospital.chegar(1, "Paciente crítico")
hospital.chegar(3, "Paciente moderado")
hospital.atender()
hospital.atender()
  
```

Briefing do Projeto Final

Na Aula 8 vocês recebem um simulador de logística. Peças obrigatórias:

- ▶ Grafo — mapa de bairros / pontos de entrega (BFS para o caminho).
- ▶ Tabela hash — dados do pacote pela chave `id`.
- ▶ Heap — ordem de saída pela urgência.
- ▶ Opcional: fila de veículos, pilha de “desfazer último despacho”, array de baús com capacidade.

Atenção

Quem chegar na Aula 8 sem hash, grafo e heap vai sofrer. Feche as práticas 4, 6 e 7. Não recomece do zero na apresentação.

Antes da próxima aula, você precisa conseguir

- ▶ Dizer o que é árvore binária completa e por que isso autoriza o array.
- ▶ Dizer se `[3, 10, 5, 11, 12, 6, 8, 15, 20, 13]` é heap — e se é min ou max.
- ▶ Inserir quatro números no min-heap e extrair em ordem crescente.
- ▶ Atender o paciente crítico antes do leve.
- ▶ Escrever no caderno qual estrutura vai para cada peça do simulador.

AULA 8

Projeto Final (PBL Integrado)

Objetivo desta aula (2h30)

Juntar as estruturas num simulador de logística, defender as escolhas em 5 a 8 minutos e entregar um README que outra pessoa consiga rodar.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em **Instalação** (pág. 7)

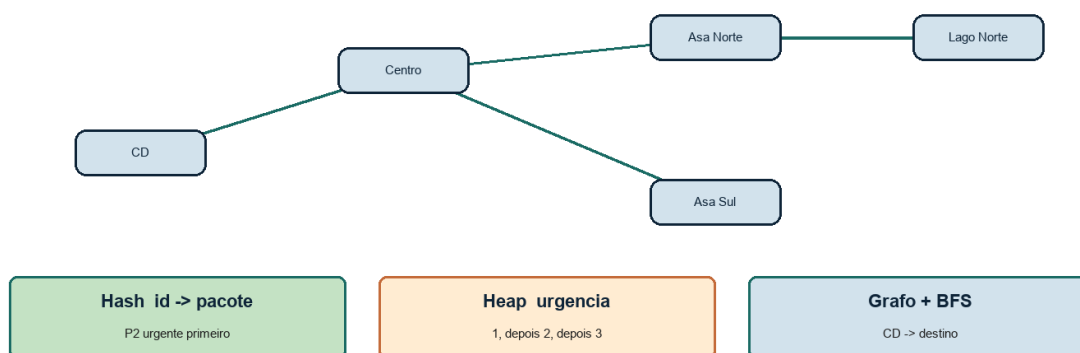
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

Atenção

Esta aula não é para inventar heap na hora. É para integrar, quebrar, consertar e apresentar.

O desafio

Há um centro de distribuição e bairros ligados por ruas (grafo). Pacotes têm id, destino e urgência. O sistema registra rotas, aceita pacotes e processa entregas do mais urgente para o menos urgente, mostrando o caminho no mapa.

Projeto final: tres pecas no mesmo sistema

Mapa (grafo) + ficha do pacote (hash) + ordem de saída (heap). P2 (urgência 1) sai primeiro.

exemplos/08_logistica.py

```
class SistemaLogistica:
    def __init__(self):
        self.rotas = Grafo()           # Aula 6
        self.pacotes = TabelaHash()    # Aula 4
        self.prioridade = MinHeap()    # Aula 7
        self.ordem = 0
```

```

def cadastrar_rota(self, origem, destino):
    self.rotas.adicionar_aresta(origem, destino)

def registrar_pacote(self, id_pacote, destino, urgencia):
    self.pacotes.inserir(id_pacote, {"destino": destino, "urgencia": urgencia})
    self.prioridade.inserir((urgencia, self.ordem, id_pacote))
    self.ordem += 1

def processar_entregas(self, deposito="CD"):
    while self.prioridade.dados:
        urgencia, _, id_pacote = self.prioridade.extrair_min()
        info = self.pacotes.buscar(id_pacote)
        caminho = self.rotas.bfs(deposito, info["destino"])
        print(f"{id_pacote} urgência {urgencia}: {caminho}")

# Monte um exemplo pequeno: 4 bairros, 3 pacotes, e rode processar_entregas.

```

O que a avaliação cobra

- ▶ Por que hash para o pacote e não uma BST (ou o contrário, se fizer sentido).
- ▶ Por que grafo no mapa — e não uma lista de endereços soltos.
- ▶ Por que heap na urgência — e não a fila FIFO da Aula 3.
- ▶ Um limite que você conhece: BST degenerada, colisão de hash, heap só no array.

Roteiro da apresentação (5 a 8 minutos)

- ▶ 1. Qual o problema? (pacotes com urgência + mapa da cidade).
- ▶ 2. Rodar o cadastro de rotas e de pacotes.
- ▶ 3. Processar entregas: o crítico sai primeiro; o caminho aparece.
- ▶ 4. No código: apontar hash, grafo/BFS e heap.
- ▶ 5. Uma frase do que você não fez de propósito (GPS real, banco, interface web).

README (entregue junto)

README.md

```

# Simulador de logística
O que é: entregas com urgência sobre um mapa em grafo.

Como o dado anda:
1. rotas (Grafo) — bairros e ruas
2. pacotes (TabelaHash) — id → destino e urgência
3. prioridade (MinHeap) — quem sai primeiro
4. BFS — caminho depósito → destino

```

```
Como rodar:  
python exemplos/08_logistica.py
```

Checklist antes de apresentar

Prática

Marque só o que o programa já faz de verdade (não o que você pretende fazer).

- ▶ ☐ Três ou mais pacotes; o de menor urgência numérica sai primeiro.
- ▶ ☐ BFS devolve um caminho que existe no grafo (teste com o desenho).
- ▶ ☐ Dá para apontar no código: hash, heap, grafo.
- ▶ ☐ README diz o que é, como rodar e o que não faz.
- ▶ ☐ Nomes de classe e arquivo fazem sentido para outra pessoa.

O que não precisa: banco de dados, site Django, API, login, mapa real de GPS.

Dica

Quer outro exemplo de união, sem ser logística? O capítulo seguinte compacta [BANANA](#) com hash, árvore e a ideia do heap — Huffman. Não entra na nota da Aula 8.

EXEMPLO PRÁTICO

Huffman junta o curso

Objetivo desta aula (2h30)

Ver um único problema (encolher **BANANA**) usando hash, árvore e a ideia do heap. Não substitui o projeto da Aula 8 e não é BST.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 7)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 8)

O simulador de logística junta hash + grafo + heap. Huffman é o *outro* exemplo de união: mesma caixa de ferramentas, outro recorte. Compactar texto. O código está em `exemplos/huffman.py`. A apostila extra (2h30) aprofunda o mesmo modelo.

O que cada aula faz aqui

Uma estrutura por papel. Se na apresentação alguém perguntar “isso é a BST?”, a resposta é não — e a tabela abaixo diz o porquê.

- ▶ *Aula 4 (hash / `dict`)* — **contar**: cada letra é chave, o valor é quantas vezes aparece. No fim, outra tabela: letra → bits (**A** → **0**). O **00** *não* vira o ASCII do A; a tabela é que diz o que cada código significa.
- ▶ *Aula 2 (lista)* — a floresta é uma listinha de árvores. Tira duas, põe o pai de volta.
- ▶ *Aula 7 (heap / prioridade)* — sempre saem os *dois menores*. Na aula usamos **sort** + **pop(0)**. No projeto, isso é o min-heap. A árvore *final* de códigos *não* é heap.
- ▶ *Aula 5 (nó esquerda/direita)* — o mesmo desenho de ponteiros. Só que o lado não compara **valor** < **no.valor**. Esquerda = bit **0**, direita = bit **1**. A letra mora na *folha*.
- ▶ *Aula 3 (fila, de longe)* — descompactar lê os bits *na ordem*, um de cada vez. Não é BFS; é descer e voltar à raiz.

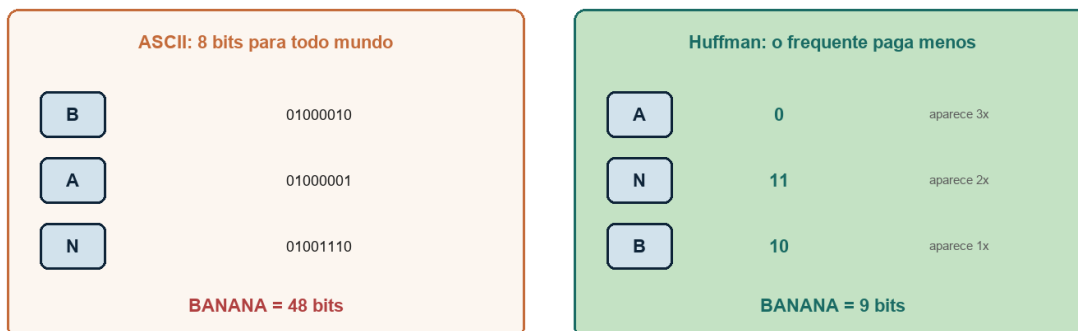
Dica

Grafo (Aula 6) *não* entra neste modelo. Huffman não é mapa de ruas. Se quiser ligar os dois, é outro problema (caminho + compactar o log da entrega).

O problema (8 bits para tudo)

BANANA tem 6 letras. Em ASCII cada uma paga 8 bits: $6 \times 8 = 48$. O **A** aparece 3 vezes, o **N** duas, o **B** uma. Huffman gasta *pouco bit no que aparece muito*.

Mesmo texto, dois jeitos de gastar bits



À esquerda, cada letra paga 8 bits. À direita, o A (o mais comum) paga 1 bit.

Na mão: contar, juntar, ler

Escreva **B A N A N A** e conte *antes* do código. Cada letra vira uma árvore de um nó (peso = frequência). Isso é a *floresta* — ainda não há uma raiz única.

Passo 1 contar BANANA

B A N A N A

Letra	Quantas vezes	Vira no da árvore
A	3	folha A, peso 3
N	2	folha N, peso 2
B	1	folha B, peso 1

$A=3, N=2, B=1$. Três árvores soltas. Huffman não insere letra numa BST.

Aula 4 — contar por chave

```
def contar(texto):
    freq = {}
    for letra in texto:
        freq[letra] = freq.get(letra, 0) + 1
    return freq
```

Passo 1. Floresta: **A(3)**, **N(2)**, **B(1)**.

Passo 2. Menores: **B(1)** e **N(2)**. Pai peso 3, esquerda B, direita N. Sobram **A(3)** e o pai **(3)**.

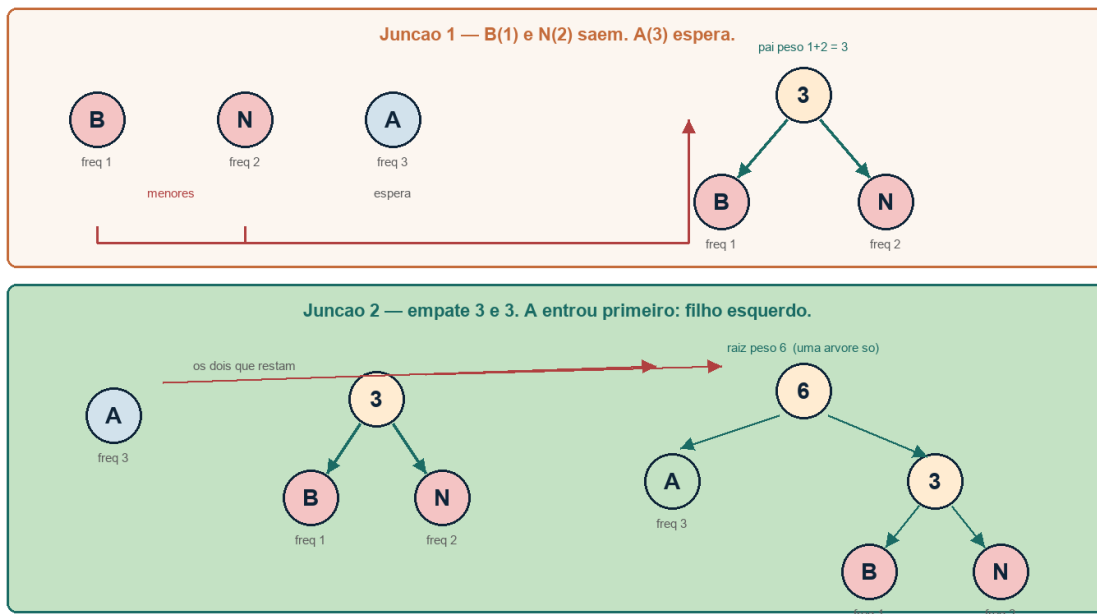
Passo 3. Empate 3 e 3. **A** tem *ordem* menor, sai primeiro, vira filho *esquerdo*. O pai **BN** vai à direita. Sobra *uma* árvore, peso 6.

Aula 7 — os dois menores (sort no lugar do heap)

```
floresta.sort(key=lambda no: (no.freq, no.ordem))
a = floresta.pop(0) # o menor
```

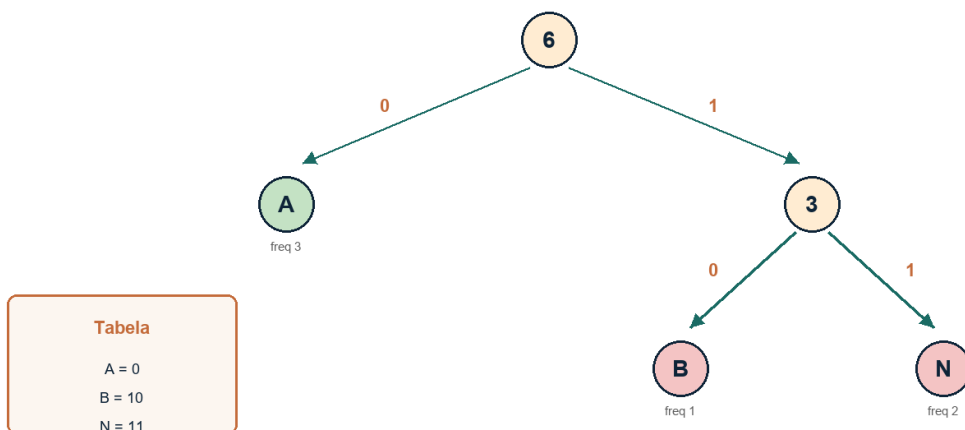
```
b = floresta.pop(0)          # o segundo menor
pai = No(a.freq + b.freq, esquerda=a, direita=b)
floresta.append(pai)
```

Passos 2 e 3 sempre junte os dois menores



Dois junções. No empate, A entra à esquerda porque a ordem é menor — não porque $A < B$.

Passo 4 esquerda = 0 direita = 1



$A = 0$, $B = 10$, $N = 11$. Folha = letra. Nó interno = só peso.

Atenção

Huffman *não* é BST. Também *não* traduz 0 para o ASCII do A (01000001). A folha guarda o símbolo A; a tabela guarda $A \rightarrow 0$. Sem a tabela (ou a árvore), 0 é só um bit.

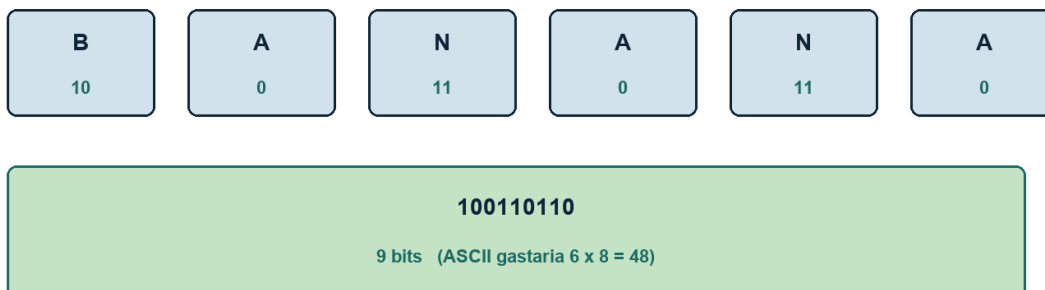
Compactar e descompactar

Compactar: para cada letra, olhe a tabela e emende os bits. Descompactar: comece na raiz, um bit de cada vez; na folha emita a letra e *volte à raiz*.

letra → bits (a tabela, não o ASCII)

```
codigos = {"A": "0", "B": "10", "N": "11"}
bits = "".join(codigos[letra] for letra in "BANANA")
# 10 + 0 + 11 + 0 + 11 + 0 -> 100110110
```

Compactar BANANA letra a letra

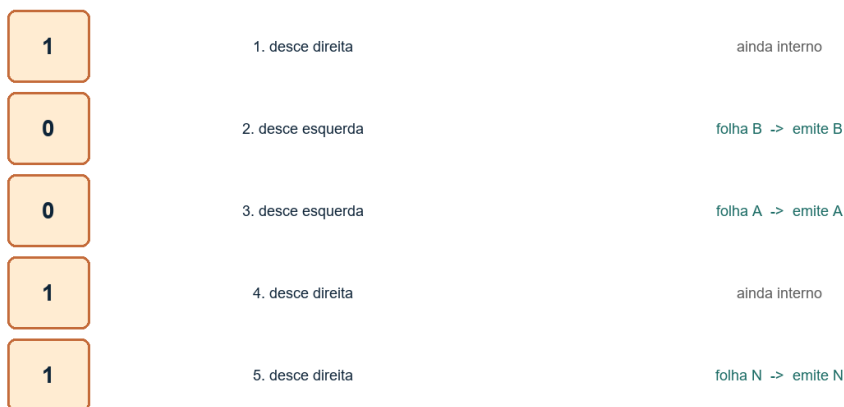


$B=10, A=0, N=11, A=0, N=11, A=0 \rightarrow 100110110$. De 48 bits para 9.

bits → letra (desce até a folha)

```
no = raiz
for bit in bits:
    no = no.esquerda if bit == "0" else no.direita
    if no. eh_folha():
        print(no.simbolo, end=" ")
    no = raiz
```

Descompactar: um bit, um passo na árvore. Folha? Emita e volte a raiz.



Depois: 0 → A, 11 → N, 0 → A. Texto: BANANA

Os dois primeiros bits 10 não são “dez”: um passo (1) e outro (0) até a folha B.

Dica

O arquivo `.huff` da aula é texto de propósito: cabeçalho `HUFFMAN`, linhas `letra código`, um `---`, depois a bitstring. Compactar e descompactar usam a *mesma* tabela. Por isso ela viaja junto com os bits.

Prática

Na mão — ABRA

Conte A, B, R. Monte a floresta, faça as junções, escreva a tabela e os bits. Rode `exemplos/huffman.py` com `texto = "ABRA"` e compare. Os bits podem diferir se o empate sair outro; o tamanho e a volta do texto é que precisam bater.

No código

Troque o texto por uma palavra da turma. Confira se `descompactar` devolve a mesma string. Abra o `.huff` e aponte: tabela (Aula 4) e árvore (Aula 5) são o mesmo mapa; o `sort` dos dois menores é o papel do heap (Aula 7).

Antes da próxima aula, você precisa conseguir

- ▶ Dizer, numa frase, o papel de hash, árvore e heap neste exemplo.
- ▶ Por que Huffman não é a BST da Aula 5.
- ▶ Por que `0` não é o ASCII do A.
- ▶ Descompactar `100110110` na árvore do BANANA, bit a bit.
- ▶ Rodar `python exemplos/huffman.py` e ver o texto voltar.

Código de referência

Os arquivos abaixo estão na pasta `exemplos/`. Rode sempre com o terminal na pasta do projeto. Este apêndice é o mapa — o código completo está nos arquivos. Funções do Python (`ord`, `deque`, etc.) estão no *Apêndice — Glossário de funções*.

- ▶ `exemplos/01_array.py` e `01_mochila.py` — array estático e PBL da mochila.
- ▶ `exemplos/02_lista.py` e `02_reprodutor.py` — lista simples e lista dupla.
- ▶ `exemplos/03_pilha_fila.py` e `03_callcenter.py` — nós LIFO/FIFO e PBL do Call Center.
- ▶ `exemplos/04_hash.py` — tabela com encadeamento.
- ▶ `exemplos/05_bst.py`, `05_bst_pendente.py`, `05_bst_rebalance.py` e `05_bst_avl.py` — BST, vareta, reset, fórmulas e os quatro casos AVL.
- ▶ `exemplos/06_grafo.py` — lista de adjacência, vocabulário e BFS.
- ▶ `exemplos/07_heap.py` e `07_triagem.py` — min-heap e prioridade.
- ▶ `exemplos/08_logistica.py` — simulador integrado (referência da Aula 8).
- ▶ `exemplos/huffman.py` — compactar BANANA (hash + árvore + dois menores).

Dica

Se o seu projeto final divergir desta referência, tudo bem — desde que você aponte no código hash, grafo e heap e o programa rode do zero.

Gabarito — Aula 4 (hash)

Atenção

Só confira depois de tentar o exercício na mão da Aula 4.

Parte A — índices (tamanho 5):

```
"u001": ord('u')+ord('0')+ord('0')+ord('1') = 117+48+48+49 = 262
262 % 5 = 2 → balde 2

"u002": 117+48+48+50 = 263
263 % 5 = 3 → balde 3

"ab": 97+98 = 195
195 % 5 = 0 → balde 0

"ba": 98+97 = 195
195 % 5 = 0 → balde 0 (colide com "ab")
```

Parte B — depois de inserir `u001` e `u002`:

```
[0] []  
[1] []  
[2] [{"u001", "João"}]  
[3] [{"u002", "Maria"}]  
[4] []
```

Parte C — exemplo com `ab` / `ba` (colisão no balde 0):

```
[0] [{"ab", "..."}, {"ba", "..."}] ← colisão  
[1] []  
[2] [{"u001", "João"}]  
[3] [{"u002", "Maria"}]  
[4] []
```

Parte D — busca mental:

- ▶ Abre *um* balde (o do índice calculado).
- ▶ Compara a *chave* de cada par (`par[0]`), não o valor.
- ▶ Tabela de tamanho `1`: tudo cai no balde `0` → vira uma lista só → busca $O(n)$.

Professor: Celso Brunno Rocha Custódio de Campos. Curso de Python — Estruturas de Dados.

Glossário de funções

Ao longo da apostila, nomes em teal sublinhado (como [ord\(\)](#)) levam até aqui. Este glossário explica só o que o curso usa — não é manual completo do Python.

Dica

No Word/PDF, clique no nome da função no texto da aula para saltar à entrada. No sumário, abra este apêndice e use a lista abaixo.

Índice rápido

- ▶ [ord\(\)](#) 66
- ▶ [sum\(\)](#) 66
- ▶ [str\(\)](#) 67
- ▶ [len\(\)](#) 67
- ▶ [append\(\)](#) 67
- ▶ [pop\(\)](#) 67
- ▶ [deque](#) 67
- ▶ [appendleft\(\)](#) 67
- ▶ [popleft\(\)](#) 68
- ▶ [setdefault\(\)](#) 68
- ▶ [%](#) 68

ord()

Recebe *um* caractere e devolve o número inteiro que o representa na tabela Unicode/ASCII. No hash didático da Aula 4, somamos esses números para transformar texto em índice.

```
print(ord("A")) # 65
print(ord("u")) # 117
print(ord("0")) # 48
```

sum()

Soma os números de uma sequência. Em `sum(ord(c) for c in chave)`, o Python percorre cada caractere, chama [ord\(\)](#) e acumula o total.

```
print(sum([1, 2, 3])) # 6
print(sum(ord(c) for c in "ab")) # 97+98 = 195
```

str()

Converte o valor para texto (`str`). No hash usamos `str(chave)` para aceitar chave que não seja string (ex.: um número) sem quebrar o `for c in ....`

```
print(str(20))      # "20"
print(str("u001"))  # "u001"
```

len()

Devolve quantos itens há na coleção (lista, `deque`, string...). No heap e nas filas, `len(self.dados)` diz o tamanho atual.

```
print(len("abc"))    # 3
print(len([10, 20])) # 2
```

append()

Método de lista: coloca um item *no final*. Em hash, cada balde é uma listinha; `baldes[indice].append([chave, valor])` acrescenta o par naquele balde. Em pilha, `append` empilha no topo (fim da lista).

```
lista = [1, 2]
lista.append(3)
print(lista) # [1, 2, 3]
```

pop()

Remove e devolve um item. Sem argumento, remove o *último* ($O(1)$ na `list`) — é o desempilhar. `lista.pop(0)` remove o primeiro, mas na `list` do Python isso é $O(n)$ (todos andam). Por isso fila usa `deque` + `popleft()`.

```
pilha = [1, 2, 3]
print(pilha.pop()) # 3
print(pilha)       # [1, 2]
```

deque

Fila de duas pontas (`collections.deque`). Entra e sai nas duas extremidades em $O(1)$. No Call Center e no BFS usamos `deque` no lugar de `list.pop(0)`.

```
from collections import deque
fila = deque(["Ana", "Bia"])
fila.append("Caio")
print(fila.popleft()) # Ana
```

appendleft()

Método do `deque`: insere no *início* em $O(1)$. No Call Center, o desfazer usa `appendleft` para a Maria voltar à frente da fila (não ao fim).

```
from collections import deque
fila = deque(["Bia", "Caio"])
fila.appendleft("Maria")
print(fila) # deque(['Maria', 'Bia', 'Caio'])
```

popleft()

Método do `deque`: remove do *início* em $O(1)$. É o “próximo da fila” (FIFO) e o passo da BFS que tira o vértice da frente.

```
from collections import deque
fila = deque(["Ana", "Bia"])
print(fila.popleft()) # Ana
```

setdefault()

Método de `dict`: se a chave *não* existe, cria com o valor padrão e devolve; se já existe, só devolve o valor atual. No grafo: `self.vizinhos.setdefault(nome, [])` garante a listinha de vizinhos sem `if` extra.

```
g = {}
g.setdefault("CD", []).append("Centro")
print(g) # {"CD": ["Centro"]}
```

%

Operador *resto da divisão* (módulo). `262 % 5` vale 2: 5 cabe 52 vezes em 262 e sobram 2. No hash, `soma % tamanho` força o índice a ficar entre 0 e `tamanho - 1`.

```
print(10 % 3) # 1
print(262 % 5) # 2
print(7 % 7) # 0
```

Professor: Celso Brunno Rocha Custódio de Campos. Curso de Python — Estruturas de Dados.